

ARCHON AGENTIC OS 2026 — VOLLSTÄNDIGE FEATURELISTE

System- & 模块-Katalog, 架构-概述 与 功能 smatrix

Stand: 11. November 2026 | 版本: v0.5.1 („Archon Hardened“) | 语言: 德语

System- & 模块-Katalog, 架构-概述 与 功能 smatrix

© Michael Keller

目录

1. 版本状态与发布概述 (v0.5.1) 3

2. 系统视图与专用工作室 (UI 视图) 3

3. Token 节省与 FinOps 生态系统 (成本最小化) 9

4. 内核、调度与守护进程架构 (Pillar 1) 12

5. 多智能体流程模型、层级与综合 (Pillar 2) 14

6. MCP-Treiber, 外部 e 服务 e & Multimodale 工具 e (Pillar 3) 16

7. 记忆, 数据库-Hygiene & Obsidian 2nd Brain (Pillar 4) 18

8. 安全、零信任治理与系统诊断 (Pillar 6) 20

9. 自进化、市场与自动文档 (Pillar 7) 21

10. Referenz-Matrix: 税 bare Evey/Hermes-Plugins 22

11. 企业与弹性架构 25

12. 自主、沙箱与治理架构 26

13. 性能核心、吞吐扩展与 Token 效率 28

14. Moderne 记忆- & RAG-架构 30

15. 可靠执行与确定性 (持久执行) 31

16. 安全、数据保护与企业护栏 33

17. 自优化、评估与群体智能 34

18. Säule 3: Wirtschaft & ERP (alpha) & 多租户套件 36

19. 架构、安全与部署特性 (v0.5.0 – v0.5.1) 36

20. 按字母排序的术语表与索引 37

1. 版本状态与发布概述 (v0.5.1)

版本	Codename	Wesentliche 架构- & Feature-里程碑 e
v0.5.1	Archon Hardened	模块 are FastAPI-Router (main.py -88%), 25 kritische + 68 hohe Security-Befunde behoben, Docker-Admin-代理 Klasse A, RBAC-用户 verwaltung (PBKDF2), 2-Phasen Systemupdate (?local_only=true), AnythingLLM 代理-集成, portabler 1-Click-Installer.
v0.5.0	Wirtschaft & ERP (alpha)	Vollständige Säule 3 (CRM/PocketBase-Sync, Artikel/Leistungen, Vorgänge ANG/AB, 发票 swesen 与 DATEV & GiroCode, Corporate-租户 en), Research Lab v2 & 55 Boardroom-框架.
v0.3.x	Sentinel / Foundation	7-Säulen OS-内核 el, 70 Spezialisten-Kader, 34+ Evey-插件 s, FinOps 3-Tier-Routing, Ephemeral Code-沙箱 en, Replay Debugging, PII-Scrubber, Semantic 缓存 & FTS5 Hybrid Search.

2. 系统视图与专用工作室 (UI 视图)

1. 视图: Mission Control (Monitoring-Cockpit & Leitstand)

Problem: Ohne zentrales Echtzeit-Cockpit fehlen Überblick 关于 aktive 智能体 en, Token-吞吐量, Tageskosten 与 blockierte 批准 n.

Mechanik: Bündelt 5 vitale KPI-Kacheln (Flottenlast, 吞吐量 TPM, 成本 wächter, Stabilität, Approval Gate), 5-Spalten-Board-项目 ion, Active Process Inspection Bar (3-Slot-Footer 针对 Heartbeat, Tool-Concurrency 与 Context Saturation), 15-Minuten-Token-活动 itätsband (SVG-Sparkline) 与 den Live-审计-Stream.

收益: *Liefert eine verzögerungsfreie 360°-监控 des gesamten 运行 ssystems auf einem einzigen 图像 schirm.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

2. 视图: Operations – Kanban Board & Task-Engine

Problem: 任务 n 在 reinen 聊天-Verläufen gehen 在…下 与 lassen sich weder 在…之后 状态 仍 在…之后 Verantwortlichkeiten steuern.

Mechanik: 5 Lebenszyklus-Spalten (To-Do, In Bearbeitung, Review & QA, Blockiert, Abgeschlossen) direkt auf kanban.db (Single Source of Truth). Unterstützt konfigurierbare WIP-Limits pro Spalte, 优先级 sfilter (P0–P3), 智能体 en-Zuweisung, 时间 erfassung 与 1-Klick-Obsidian-Verknüpfung. Integriert 4-Augen 质量 s-Gate: Vor Verschieben einer 任务 auf 'done' prüft ein Peer-Reviewer (Katharina Völk 针对 QM/Compliance 或 Robert) Arbeitsergebnis auf Vollständigkeit, Substanz 与 Handlungsfähigkeit (1–100% Score). Bei 'changes_requested' greift eine 状态-Sperre; ein Human-in-the-Loop Override (force=True) übersteuert 在 Bedarf.

收益: *Strukturiert 所有 operativen 项目 e transparent 与 verhindert Systemüberlastung 通过 Work-in-Progress-Schranken.* Verhindert unfertige Arbeitsstände 在 'done'-状态 通过 zwingendes Vier-Augen-Review.

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内

des 模块 s).

3. 视图: Operations – Interaktives 2D-Organigramm & 报告 slinien

Problem: Flache Listen 的 智能体 en zeigen 无 Weisungsbefugnisse 或 升级 spfade 在 blockierten 流程 en.

Mechanik: Visualisiert 56 节点 与 55 报告 slinien 作为 interaktiven 2D-Graphen (Apex -> CEO Robert -> Fach-Leads -> Spezialisten). Unterstützt Drag & Drop 至 Umhängen 的 Vorgesetzten, Raster-Ausrichtung, 团队-Karten-视图 与 dauerhafte 存储 ung 在 config/org_hierarchy.json.

收益: *Definiert verbindliche 升级 s- 与 Delegationsketten, denen System 在 Blockaden 自动 folgt.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

4. 视图: Operations – 内部 e 智能体 en-Kommunikation (Funkverkehr)

Problem: Wenn Spezialisten untereinander Teilaufgaben übergeben, überflutet dies entweder den Nutzer-聊天 或 bleibt unsichtbar.

Mechanik: Dedizierte Abhörzentrale 与 渠道 liste (#allgemein, #tech-dev, #legal, #marketing) 与 chronologischem 消息 en-Feed aller Inter-智能体 en-Absprachen aus kanban.db.

收益: *Macht Zusammenarbeit 智能体 en-Flotte nachvollziehbar, 无 primäre Nutzer-Postfach zu belasten.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

5. 视图: Eingang & Triage (3-Spalten HITL-Gatekeeper & Postfach)

Problem: Unkontrollierte Schreibaktionen 的 智能体 en (z. B. 电子邮件-Versand) sowie verstreute Eingänge aus Mail 与 Telegram bergen 安全 srisiken.

Mechanik: Dreispaltige Triage-Zentrale: Spalte 1 针对 gelb markierte Human-in-the-Loop-Freigaben (Freigeben/Ablehnen), Spalte 2 针对 IMAP/SMTP-E-Mail-Synchronisation 与 Auto-Triage & Delegation, Spalte 3 针对 den gespiegelten Live-Telegram-Stream (@Beatman_bot).

收益: *Stellt sicher, dass 无 externe Schreiboperation 无 ausdrückliche 批准 des Betreibers ausgeführt wird.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

6. 视图: 智能体团队 & Flottenmanagement (8-Reiter-Cockpit)

Problem: Spezialisten benötigen individuelle 模型 e, System-Prompts, 工具 e 与 作业指导 en an einem zentralen Ort.

Mechanik: Verwaltet 70 Spezialisten-个人资料 e 与 快速 filtern (活动, Favoriten, Alle, 7 Domänen), Hot-Swap 针对 Provider & 模型 sowie 8 Fach-Reitern: 概述 (360°-Dossier & Telegram-Rufname), Direkt-聊天, 能力 en (Skills), Operations (Task-Historie), SOUL (System-Prompt-Editor), Personalakte (SOPs), Talent-市场 platz 与 Skill-市场 platz.

收益: Zentrale Personalabteilung 至 sekundenschnellen Kalibrierung 与 扩展 des gesamten 智能体 en-Stamms.

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

7. 视图: Master 聊天 (Dual-Mode Workspace & Multi-Session-Sidebar)

Problem: Beim direkten Arbeiten 与 智能体 en fehlte eine fokussierte Vollbild-环境 与 schnellem Wechsel 之间 1:1-对话, 组 nchat 与 Missions-订单.

Mechanik: Bietet den Dual-Mode-Switch ('Direkt' 针对 sofortige 回答 vs. 'Mission' 针对 strukturierte Task-设备), Multi-File-Upload (bis 5 文件 en), Browser-Screen-Capture (Kamera-Button), 语言 ingabe (Diktat), Reasoning-预算-Wähler (0 bis 16k Tokens), KI-Prompt-Optimizer 与 rechte Session-Historie (智能体 en vs. 组). Beinhaltet Telegram Voice-Pipeline: Eingehende Sprachnachrichten (.ogg/Opus) 的 unterwegs 将 via Whisper/Gemini transkribiert, anhand 的 Rufnamen (/robert, /anwalt etc.) an den Fachagenten geroutet 与 回答 en via edge-tts (de-DE-Conrad 新建 ral) 作为 natürliche Sprachmemos direkt auf Telegram zurückgesendet.

收益: *Vereint 高 geschwindigkeits-对话, multimodale 输入 与 Missions-控制 在 einer durchgängigen Arbeitsfläche.* Ermöglicht vollständige mobile Sprachbedienung 与 bidirektionalen 音频-Austausch 关于 Telegram.

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

8. 视图: AI Mastermind & Boardroom (55 Methodik-框架)

Problem: Komplexe strategische 决策 en erfordern strukturierte Management-方法 n 与 kontroverse Perspektiven statt Einzelmeinungen.

Mechanik: Simuliert mehrstufige Gremiensitzungen (Chairman + 3 reale Fachagenten-Slots + Red-团队-Challenger) anhand 的 34 auswählbaren 框架 在 7 类别 n (35 Base) (z. B. CLEAR, Design Sprint, Double Diamond, Lean 开始 up, First Principles, Game Theory, Blue Ocean, Pre-Mortem, Six Sigma).

收益: Erzwingt methodische Strenge, deckt blinde Flecken 通过 Red-团队 auf 与 liefert entscheidungsreife Board-Memos.

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

9. 视图: Showroom (项目-Explorer, Artefakt-Galerie & Python-沙箱)

Problem: Generierte 报告 e, HTML-Prototypen, Kalkulationen 与 Skripte gehen 在 langen Konversationen verloren.

Mechanik: Katalogisiert 所有 Arbeitsergebnisse 在…之后 Ersteller, 日期 与 Typ. Beinhaltet den Workspace Vault Reader 与 Live-Rendering 针对 Markdown/HTML/JSON sowie eine isolierte Python-沙箱 至 sicheren Code-执行. 高级 um den interaktiven Showroom Live-Runner: Ermöglicht direkte 执行 的 Python- 与 Code-文件 en 在 Browser 与 12s Timeout, isoliertem Scratch-目录 与 ausklappbarem 日期 al-Drawer (STDOUT, STDERR, Exit-Code 与 Millisekunden-期限).

收益: *Dient 作为 验收- 与 Präsentationshalle 针对 所有 fertigen Deliverables 在…之前 dem 客户 n- 或 生产 seinsatz.* Code-Artefakte 与 Skripte lassen sich 无 Konsolenwechsel direkt 在 Showroom verifizieren 与 testen.

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

10. 视图: 中央图书馆 & 2nd Brain (RAG, Auto-OCR & Obsidian-Vault)

Problem: Fachbücher, DIN-标准 en 与 Bebauungspläne 作为 PDF konnten 的 智能体 en bisher 不 无 manuelles Kopieren durchsucht werden.

Mechanik: Vereint 3 Sub-Tabs 与 4 Core Hubs (00_HR_智能体 en, 02_知识 sbibliothek, 日志 e/, 00_Notebook_Synthesen): 1. AnythingLLM RAG-Index (Port 3001), 2. Dokumenten-Erfasser 与 Drag & Drop, Auto-OCR (Docling) 与 KI-Themenklassifikation, 3. Direkter Obsidian-文件 baum & Markdown-Viewer inklusive Vault-Umschalter.

收益: Verwandelt PDF-鉴定 vollautomatisch 在 durchsuchbares Markdown- 与 Vektor-知识 针对 所有 70 智能体 en.

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

11. 视图: 工作流-Engine (n8n Automation Mesh auf Port 5678)

Problem: Wiederkehrende 接口 nprozesse (Cron-Jobs, Webhooks, 数据库-Abgleiche) verbrauchen unnötig Tokens, wenn sie 关于 LLMs laufen.

Mechanik: Betten den visuellen n8n-工作流-Editor direkt ein 与 verbindet ihn 关于 den Skill n8n-automation-mesh (FastMCP) 与 dem CEO-智能体 en Robert. 活动 ierte 0-Token-Routinen: 1. Morning-Briefing (werktags 07:30 Uhr 与 Obsidian-Ablage 与 Telegram-Zustellung), 2. Weekly Maintenance (sonntags 23:00 Uhr 与 SQLite-备份 & Health-审计), 3. RSS & News Watchdog (stündlicher Scan), plus 5 aktive 功能 s-Webhooks 针对 Web-Extraktion, Messaging, Dokumenten-Rendering, Feed-搜索 与 CRM-Sync.

收益: *Kombiniert deterministische 0-Token-后台进程 e 与 kognitiven 智能体 en-决策 en an 密钥 stellen.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

12. 视图: LLM 网关 (OmniRoute AI Router 仪表板)

Problem: Die feste Bindung an einen einzigen Cloud-Anbieter führt 在 Rate-Limits 或 Ausfällen 至 sofortigen Systemstillstand.

Mechanik: Zentrales 代理-仪表板 (Port 20128 / 8000) 与 Multi-Provider-Vault, Echtzeit-延迟 messung (ms), HTTP-错误 monitor (429/500), automatischem Circuit-Breaker 与 4-stufiger Fallback-Kaskade (GLM-4.7 Flash -> Gemini 2.5 Flash -> Claude 3.5 Sonnet -> lokales Llama 3.3 70B).

收益: *Sichert 99,9 % 可用性 aller kognitiven 流程 e 在 minimalen API-成本.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内

des 模块 s).

13. 视图: Token Guard (Guardian & Token Analytics 在 3 Sub-Views)

Problem: Fehlende 透明度 darüber, welcher 智能体, welches 模型 或 welche Einzelsitzung meisten Tokens 与 成本 verursacht.

Mechanik: Bietet 时间 filter (24h, 7d, 30d, Gesamt), 非活动 itäts-Timer 与 3 Sub-Views: 1. 概述 & Tages-趋势-Chart 与 个人资料-/Modell-Tabellen 与 保护-Schranken-表单, 2. Einzel-Session Verbrauchs-Inspektor (Input, Output, 缓存-Read Tokens sortierbar), 3. 资源 n & Task-Drosselung (Schieberegler 1–16 Tasks).

收益: *Volle 成本 kontrolle bis auf einzelne Session-ID herab inklusive sofortiger 预算- 与 Parallelitäts-Begrenzung.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

14. 视图: 扩展 en & 模块-Zentrale (34 Evey-Spezial-插件 s)

Problem: 安全 s-, 存储- 与 分析 funktionen ließen sich bisher 不 modular pro Fachgebiet zu- 或 abschalten.

Mechanik: 税 t 34 spezialisierte Hermes/Evey-Plugins 关于 8 Fach-Reiter: 安全 & Wächter (4), Performance & 缓存 (2), Monitoring & Analytics (3), 智能体 en-Intelligenz (6), 规划 & Tasks (2), Brücken & 接口 n (4), 记忆 & 知识 (3), Tools & Execution (3) plus Finanzen/Qualität/Wartung. Beinhaltet standardisierte 插件 system & Runtime-Hooks: 模块 are 文件夹 struktur (plugins/<plugin_id>/plugin.json), 4 Lifecycle-Hooks (on_load, on_enable, on_disable, on_unload) 与 2 Tool-Interzeptoren (before_tool_execute, after_tool_execute) 与 vorinstallierten 开始 er-插件 s (System Metrics, Security 审计 Hook, Cost Guard).

收益: *Erlaubt präzise Zu- 与 Abschalten einzelner Systemfähigkeiten 在 laufenden 运行 per Mausklick.* Ermöglicht isolierte Runtime-扩展 en 与 globale Tool-Interzeption 无 Eingriff 在 den 内核 el-Code.

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

15. 视图: Research Lab (Deep Search & Evidenz-Triangulation)

Problem: 简单 e Suchanfragen liefern ungeprüfte Einzeltreffer 无 来源 nabgleich 或 Vertrauensbewertung.

Mechanik: Mehrstufiges Forschungs-Cockpit 与 wählbarer Suchtiefe, 来源 n-Monitor, Evidenz-Score, Live-Synthese-日志 至 Triangulation widersprüchlicher Aussagen 与 1-Klick-导出 在 den Obsidian-Vault. Beinhaltet Autonome Tiefenrecherche & Reflexionsschleife v2 (Re-Search Loop): Zerlegung komplexer Recherchen 在 3–5 Teilfragen, autonome Lückenanalyse, bis zu 3 iterative Nachfolge-Suchläufe, Multi-Provider Fallback (SearXNG / Jina Reader) 与 SSRF-保护.

收益: *Erzeugt belastbare, zitierfähige 市场-, 法律 s- 与 Technik-Dossiers.* Schließt 知识 slücken selbstständig 通过 rekursive Folgerecherchen 在 widersprüchlichen 或 unvollständigen 来源 n.

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

16. 视图: Telemetrie & Logs (3-来源 n Live-审计 Stream)

Problem: Bei 错误 n 在 容器 n 或 智能体 en-Prompts musste bisher mühsam 在 日期 al 关于 Docker-Befehle gesucht werden.
Mechanik: Bündelt 3 Log-来源 n 在 Echtzeit: Physische Docker-容器-Logs (stdout/stderr 11 容器), kognitive 智能体 en-Logs 与 den kryptografischen 审计-Trail. Mit Level-筛选 (DEBUG, INFO, WARN, ERROR), Auto-Scroll-Lock 与 导出.

收益: *Unbestechlicher Flugschreiber 针对 sekundenschnelle 错误 diagnose 与 合规-证明 e.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

17. Spezial-Studio: Archon Prompt Studio (832 模板 n & Fabric SOP)

Problem: Unpräzise formulierte 任务 n führen zu Halluzinationen, falschen 输出 formaten 与 teuren Korrekturschleifen.
Mechanik: Integrierter Baukasten 与 824 Enterprise-模板 n 在 16 Fachkategorien, 关于 255 Fabric-SOP-Mustern 与 5-Felder-Formel (1. 角色, 2. Kontext, 3. 任务, 4. Format, 5. Constraints/Negativ-Regeln) sowie KI-Prompt-Optimizer.
收益: Zwingt 每 Delegation 在 eine glasklare Struktur 与 überträgt sie per Klick 在 聊天 或 Kanban-任务.

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

18. Spezial-Studio: Visual CSS Live Studio & In-Place Transform Gizmo (Alt + S)

Problem: Design-Anpassungen erforderten bisher 搜索 n 在 CSS-文件 en 与 ständiges 新建 laden des Browsers.
Mechanik: Schwebendes Echtzeit-Design-Studio (Alt + S) 与 Fadenkreuz-DOM-Inspektor, interaktivem Transform-Gizmo (Position ziehen, 4 Ecken/Kanten skalieren, Eckenrundung ziehen), Theme-个人资料-Manager 与 direktem CSS-导出 在…之后 custom_studio.css.

收益: *Ermöglicht pixelgenaues Live-Styling gesamten 用户 oberfläche 在 laufenden 运行 无 页 n-Reload.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

19. Spezial-Studio: Design System Lab, AppShell Architekt & 15-组件 n-Studio

Problem: Abweichende Header-Höhen, Abstände 或 Kontraste 之间 Hell- 与 Dunkelmodus stören visuelle Ruhe.
Mechanik: Beinhaltet den interaktiven AppShell-Architekten (56px Header-Flucht, Sidebar-Breite, Viewport-Padding 与 Live-Miniatursimulation) sowie 15-组件 n-Design-Studio (os-workspace bis os-badge) 与 Dual-Theme-Engine (Light/Dark).

收益: *担保 rt ein einheitliches, blendfreies B2B-SaaS-Erscheinungsbild 关于 所有 20 视图 en hinweg.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert

(vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

20. Spezial-视图 en: System Control Center (11 Fachreiter, Live-Dock) & Globale Omnibox (Strg + K)

Problem: 设置 en waren verstreut; Navigation 之间 20 视图 en 与 70 智能体 en erforderte viele Mausklicks.

Mechanik: Das System Control Center bündelt 7 Administrations-Reiter (Hermes Core inkl. YAML-Editor, 容器 & 服务 e, Provider & 网关, Methodiken, 插件 s, Kommunikation, Design) 与 lässt sich per 'Live-Dock' rechts andocken. Die Omnibox (Strg + K) navigiert per Tastatur sofort zu jedem Tab (#), 智能体 en (@) 或 Befehl. Integriert 1-Klick 更新 system & Continuous Deployment: 自动 er GitHub-Sync 与 Pre-更新 Snapshots aller SQLite-数据库 en (WAL) 在 data/snapshots/ 在…之前 jedem Git-Pull, 1-Klick Rollback 与 minimalen Git-Stat-检验 en gegen Mount-Timeouts.

收益: Maximale Bediengeschwindigkeit 针对 Administratoren 与 Power-User. Ermöglicht gefahrlose Software-更新 s 与 automatischem 数据库-Rollback 在 错误 fall.

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

3. Token 节省与 FinOps 生态系统 (成本最小化)

21. 3-Tier 自动 e 成本-Routing-Caps (LOW / MEDIUM / HIGH)

Problem: Wenn einfache Routineaufgaben auf teuren Spitzenmodellen (wie Claude Sonnet 或 GPT-4o) laufen, explodieren Tageskosten.

Mechanik: Konfigurierbare Preis-Obergrenzen pro 1 Million Tokens 在 System Control Center: LOW (max. $0.10/1M -> erzwingt Free-Tier/Flash-Lite), MEDIUM (max. $1.00/1M -> Haiku/Gemini Flash), HIGH (No Cap -> beste Reasoning-模型 e 针对 komplexe 架构).

收益: *Senkt laufenden API-输出 n 在 Alltag um bis zu 85 %, 无 在 schwierigen 任务 n an 质量 einzubüßen.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

22. FreeLLM-API Docker-代理 & 'auto/best-free' Routing (Port 3002)

Problem: 成本 lose Kontingente verschiedener Anbieter 是 schwer 手动 zu überwachen 与 wechseln häufig.

Mechanik: Ein dedizierter Docker-容器 (freellmapi auf Port 3002) bündelt kostenfreie LLM-Endpunkte. Das Routing-个人资料 'auto/best-free' wählt 自动 stärkste verfügbare Null-Euro-模型.

收益: *Ermöglicht den 持续时间 betrieb 的 Hintergrund- 与 Triage-智能体 en zu exakt $0.00 API-成本.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

23. Stufenlose Reasoning-预算-控制 在 Master 聊天 & Router (0 bis 16k Tokens)

Problem: Reasoning-模型 e (Thinking-Mode) verbrauchen oft tausende unsichtbare Denk-Tokens 针对 triviale 问题 n.

Mechanik: 6-stufiger Regler (Aus: 0 Tok, Minimal: ~1k, 低: ~2k, 中: ~4k, 高: ~8k, Maximal: ~16k) 在 聊天-Footer 与 在 agent_router.py.

收益: *Schaltet teure Denk-Schleifen 在 einfachen 任务 n komplett ab 与 reserviert tiefes Reasoning 针对 echte Komplexfälle.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

24. Evey-缓存 & 缓存-TTL-控制 (24h Semantischer & Exakter 回答-缓存)

Problem: Wiederkehrende identische 或 很 ähnliche Systemabfragen führten 每 Mal zu neuen, kostenpflichtigen API-Calls.

Mechanik: Das 插件 evey-cache speichert 模型 antworten 与 konfigurierbarer Lebensdauer (默认: 24 Stunden TTL) 与 weist 在 Session-Inspektor eingesparten 缓存-Read-Tokens separat aus.

收益: *Beantwortet wiederholte 查询 n 在 0 Millisekunden zu 0 Token-成本.*

Preis & Schattenseite (Trade-off): Ist 缓存-TTL zu hoch (> 72h), liefert System 在 dynamischen Themen veraltete 回答 en.

Wann ausschalten? & 阈值 e: Ausschalten 在 Echtzeit-查询 n (Live-Kurse, 服务器 status). Std-TTL: 24h.

25. Max-Context & Max-History Hard-Clipper 在 Guardian (32k / 20 Turns)

Problem: Endlos wachsende 聊天-会话 en senden 在 每 neuen 问题 gesamte Historie erneut 与 与 treiben Input-Tokens exponentiell 在…之后 oben.

Mechanik: Im Guardian einstellbare Hardschranken 针对 maximale Kontextfenster (z. B. 32.000 Tokens) 与 maximale Anzahl übermittelter 聊天-消息 en (z. B. letzte 20 Turns).

收益: *Kapselt maximal möglichen Input-成本 pro Turn verlässlich 在…之后 oben ab.*

Preis & Schattenseite (Trade-off): Schneidet 消息 en oberhalb 的 32k Tokens 或 20 Turns hart ab, wodurch 很 früher Kontext wegfällt.

Wann ausschalten? & 阈值 e: Für tiefe Dokumenten-分析 n auf 64k–128k 在 Hermes Core anheben.

26. Dynamischer Token-Clipper 针对 Low-Context-模型 e (backend/token_clipper.py)

Problem: Kompakte 或 lokale 模型 e (8k/16k) brechen ab, wenn große Tool-输出 n Kontextfenster sprengen.

Mechanik: Bewahrt zwingend den System-Prompt (Index 0) 与 letzte 用户 anweisung, kürzt überlange Einzelblöcke intelligent 与 entfernt ältere Zwischenschritte.

收益: *Erlaubt den stabilen Einsatz extrem günstiger Kleinmodelle 也 在 umfangreichen 任务 n.*

Preis & Schattenseite (Trade-off): Kürzt überlange Tool-输出 n 在 Mitte heraus ([...SNIP...]), wodurch einzelne Code-Zeilen 在 Rohtext fehlen können.

Wann ausschalten? & 阈值 e: Kurzzeitig deaktivieren, wenn komplette Roh-Logs ungekürzt benötigt werden.

27. Sliding Window Memory Compression (Konsolidierter 决策 skontext)

Problem: Beim einfachen Abschneiden alter 消息 en gehen frühere 项目 beschlüsse verloren.

Mechanik: Verdichtet ältere Gesprächsrunden zu einer kompakten 总结 与 injiziert 本 作为 Block [KONSOLIDIERTER GESPRÄCHS- & ENTSCHEIDUNGSKONTEXT] auf Position 1.

收益: *Spart bis zu 70 % historische Konversations-Tokens 在 vollem Erhalt aller getroffenen 决策 en.*

Preis & Schattenseite (Trade-off): Der Komprimierungs-步骤 kostet einmalig ~800 Tokens 与 glättet wörtliche Formulierungen früher Runden.

Wann ausschalten? & 阈值 e: Bei Bedarf 关于 protect_last_n (Std: 20) 在 Hermes Core feinjustieren.

28. LRU-K Page Swapping 针对 Subagenten-RAM (backend/context_swapper.py)

Problem: Parallele Subagenten halten riesige Mengen an Rohdaten gleichzeitig 在 aktiven Prompt-存储.

Mechanik: Lagert 不 更多 akut benötigte Kontext-页 n ab 80 % 利用率 在…之后 dem Least-Recently-Used-Prinzip 在 SQLite aus 与 lädt sie 仅 在 Bedarf (Re-Hydration) zurück.

收益: *Hält Prompts schlank 与 verhindert unnötige Token-Gebühren 针对 inaktives Hintergrundwissen.*

Preis & Schattenseite (Trade-off): Das Auslagern 与 Nachladen 的 智能体 en-RAM auf Disk erzeugt ~10–25 ms I/O-Latenz 在 Kontextwechsel.

Wann ausschalten? & 阈值 e: Ab 80 % RAM-利用率 aktiv lassen, um OOM-Abstürze zu verhindern.

29. JEV Router — 0-Token Deterministische Intent-Vorfilterung (jev_router.py)

Problem: Jede einfache Systemanweisung 关于 ein großes Sprachmodell zu klassifizieren, verschwendet Tokens 与 时间.

Mechanik: Klassifiziert eingehende 输入 n 在…之前 dem LLM-Aufruf per deterministischem JSON Execution Vector 与 Keyword-Heuristik.

收益: *Leitet klare Systembefehle direkt an 目标 routine weiter, 无 ein einziges LLM-Token zu verbrauchen.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

30. ASD-STE100 Simplified Technical English Prompter (.agents/skills/asd-ste100-prompter)

Problem: Weitschweifige Prosatexte 在 System-Prompts 与 智能体 en-Anweisungen blähen jeden einzelnen API-Call unnötig auf.

Mechanik: Transformiert Instruktionen 在…之后 dem Luftfahrt-默认 ASD-STE100 (eine Bedeutung pro Wort, aktive Verben, kurze Sätze, null Füllwörter).

收益: *Reduziert Token-Länge 的 System-Prompts um ca. 30–40 % 与 senkt gleichzeitig 错误 rate 在 Tool-Calls.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

31. Anti-AI-Slop & Output-Komprimierung (.agents/skills/anti-ai-slop)

Problem: Sprachmodelle erzeugen standardmäßig lange Höflichkeitsfloskeln, Einleitungen 与 redundante 总结 en (teure Output-Tokens).

Mechanik: Unterbindet KI-Floskeln, leere Phrasen 与 künstliches Aufblähen direkt 在 System-Prompt 与 bereinigt 输出 n.

收益: *Spart bis zu 25 % besonders teuren Output-Tokens 与 liefert sofort verwertbare Fachtexte.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

32. Pre-Flight 成本-Schätzer & 预算-预测 (backend/cost_estimator.py)

Problem: Vor dem 开始 großer 任务 n war 不 absehbar, was Lauf kosten wird.

Mechanik: Berechnet vorab 关于 POST /api/router/estimate-cost erwarteten Input-/Output-Tokens sowie den Dollar-Betrag anhand aktuellen 模型-Preisliste 与 gleicht ihn 与 dem Task-预算 ab.

收益: *Warnt 在…之前 teuren Läufen, 仍 bevor erste Cent ausgegeben wird.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

33. Dynamisches Task-预算 与 80%-Spar-Downgrade & 100%-Stopp

Problem: Ein einzelner komplexer Task konnte unbemerkt gesamte Tagesbudget verzehren.

Mechanik: Überwacht kumulierten 成本 pro 任务 (z. B. Limit $0.25): Schaltet 在 80 % 自动 auf ein günstiges Spar-模型 herunter 与 pausiert den Task 在 100 % 与 Telegram-批准 anfrage.

收益: *Verhindert 成本 ausreißer 在 mehrstufigen autonomen Recherchen.*

Preis & Schattenseite (Trade-off): Bei Erreichen 的 80 % des Task-预算 s 将 auf ein günstigeres Spar-模型 heruntergeschaltet; 在 100 % erfolgt Hard-Stop.

Wann ausschalten? & 阈值 e: Task-预算 (Std: 0,50 $) 针对 große 架构-Läufe auf 2,00 $ anheben.

34. Loop-Killer & Max-Steps-Begrenzung (15 步骤 e / Jaccard > 90%)

Problem: Wenn ein 智能体 在 einem fehlerhaften Tool-Aufruf 在 eine Endlosschleife gerät, verbrennt er 在 Sekundentakt Guthaben.

Mechanik: Prüft jeden 步骤 auf exakte Wiederholungen (>= 3) 或 semantische Kreisbezüge (> 90 % Ähnlichkeit) 与 kappt 任务 n spätestens 在…之后 dem konfigurierbaren Max-Steps-Limit (默认: 15 步骤 e).

收益: *Schützt 账户 zuverlässig 在…之前 unbemerkten Endlosschleifen 关于 Nacht.*

Preis & Schattenseite (Trade-off): Zu scharfe Grenzwerte (Max Identisch = 2 或 Jaccard < 0.80) stoppen legitime iterative 任务 n 如 schrittweises 测试-Fixing 或 Paginierung zu früh.

Wann ausschalten? & 阈值 e: Kurzzeitig ausschalten, wenn ein 智能体 bewusst viele gleichförmige Schleifen-步骤 e (z. B. Massen-迁移 的 50 文件 en) abarbeiten soll. | 阈值 e: Max Identisch: 2 = stoppt Retries | > 5 = teure Schleifen (Std: 3). Jaccard: < 0.80 = Fehlalarme | > 0.96 = erkennt Schleifen spät (Std: 0.90). Max 步骤 e: Std 15.

4. 内核、调度与守护进程架构 (Pillar 1)

35. Hermes Master Orchestrator (Hermes-Core)
Problem: Unkoordinierte Einzelagenten arbeiten gegeneinander 或 verlieren Gesamtziel aus den Augen.
Mechanik: Zentrale 流程-Leitstelle 至 订单 sanalyse, 角色 nauswahl, 监控 与 结果-验收.
收益: Sichert strategische Kohärenz 关于 所有 70 Spezialisten.

Preis & Schattenseite (Trade-off): Mehr aktive Hintergrund-服务 e 与 hohes Reasoning ('high') erhöhen Grundlast 与 Token-成本.

Wann ausschalten? & 阈值 e: Über 14 Sektionen 在 Tab 'HERMES CORE' granular steuerbar.

36. Tickless 内核 el 守护进程 & Asyncio Event-Loop (Port 8420 / 8501)

Problem: Ständiges Polling 在 Sekundentakt verschwendet CPU-资源 n 与 Systemenergie.

Mechanik: Berechnet den exakten 时间 punkt des nächsten fälligen Tasks ('Sleep-until-next-due') 与 versetzt den 内核 el bis dahin 在 den Ruhezustand.

收益: *Minimale Systemlast 在 Leerlauf 在 verzögerungsfreier Aufwach-Reaktion.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

37. 优先级 s-Preemption Scheduler (P0–P3 队列)

Problem: Dringende 用户 fragen 在 Direkt-聊天 mussten warten, wenn Hintergrund-Jobs 所有 Slots belegten.

Mechanik: Priorisierte 队列: P3-任务 n (Direkt-聊天 / 紧急情况) überholen laufende P1/P2-Routinejobs sofort.

收益: *担保 rt schnelle 回答 zeiten 在 对话 trotz hoher Hintergrundlast.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

38. Frei skalierbare Concurrency-Drosselung (backend/concurrency_throttle.py)

Problem: Zu viele gleichzeitige 智能体 en führen zu HTTP-429-Rate-Limits 与 99 % CPU-Spitzen.

Mechanik: Reguliert aktive Worker 关于 eine Asyncio-Semaphore (einstellbar per Slider 的 1 bis 16 parallelen Tasks 在 Guardian 与 Control Center).

收益: *Erlaubt gedrosselten 保护 betrieb auf Laptops ebenso 如 高 lastbetrieb auf Workstations.*

Preis & Schattenseite (Trade-off): Knappe Stagnations-Limits (< 30 Sek.) brechen langsame Deep-Reasoning-模型 e (z. B. o3/R1) fälschlich mitten 在 Denkprozess ab; 更多 Worker erhöhen RAM- 与 API-Spitzenlast.

Wann ausschalten? & 阈值 e: Ausschalten 在 extrem langlaufenden Einzel-分析 n 或 lokalen LLM-Läufen auf langsamer Hardware. | 阈值 e: Max Worker: 1–2 = Rückstau | > 8 = RAM-Spitzen & HTTP-429 (Std: 4). Stagnation: < 30s = bricht Denker ab | > 180s = späte Heilung (Std: 45s).

39. Autonomer 30-Sekunden-Watchdog & Crash-Recovery (watchdog_daemon.py)

Problem: Abgestürzte 或 eingefrorene Tasks blieben dauerhaft 在 状态 in_progress hängen.

Mechanik: Prüft 所有 30 Sekunden sämtliche laufenden 流程 e, bricht Zombie-Tasks 在…之后 Timeout (120s) ab 与 überführt verwaiste 任务 n 在 automatische Crash-Recovery.

收益: *Hält 所有 Worker-Slots frei 的 Karteileichen.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

40. Deadlock- & Stagnations-Supervisor (backend/deadlock_supervisor.py)

Problem: Gegenseitige Abhängigkeiten 之间 zwei 智能体 en 可 zu einem stillen Einfrieren führen.

Mechanik: Erkennt zirkuläre Wartezustände 与 Tasks 无 Fortschritt (> 45s) 与 löst Blockade autonom 与 exponentiellem Backoff auf.

收益: *Verhindert Pipeline-Stillstände 在 komplexen 组 narbeiten.*

Preis & Schattenseite (Trade-off): Ein zu knappes Stagnations-Limit (< 30s) bricht langsame Deep-Reasoning-模型 e (o3/R1) mitten 在 Denkprozess ab.

Wann ausschalten? & 阈值 e: Bei Nutzung 的 Reasoning-模型 en Stagnations-Limit auf 90–120s erhöhen (Std: 45s).

41. Flotten-Selbstheilung & 容器-Auto-Restart (backend/fleet_self_healing.py)

Problem: Fällt einer 11 Docker-容器 aus, musste er 手动 新 gestartet werden.

Mechanik: Überwacht 所有 Core-容器 关于 den abgesicherten docker-socket-proxy 与 führt 在 Ausfällen (> 60s) autonom einen 新建 start inklusive 审计-Eintrag durch.

收益: *Sichert den unbeaufsichtigten 24/7-Betrieb gesamten Docker-Landschaft.*

Preis & Schattenseite (Trade-off): Ein dauerhaft defekter 容器 可 通过 Auto-Restart 在 eine 新建 start-Schleife geraten.

Wann ausschalten? & 阈值 e: Während manueller 容器-维护 sarbeiten kurzzeitig deaktivieren.

42. Proaktiver 明天-Standup & Weckdienst (backend/autonomous_standup.py)

Problem: 明天 s fehlte eine kompakte 概述 关于 nächtliche Läufe, 成本 与 anstehende 决策 en.

Mechanik: Generiert täglich um 08:00 Uhr einen konsolidierten 状态 bericht 在 Obsidian-Vault (01_PROJEKTE/Daily_Briefings/) 与 pusht ihn an 仪表板 与 Telegram.

收益: *开始 et jeden Arbeitstag 与 einer fertigen 决策 svorlage.*

Preis & Schattenseite (Trade-off): Jedes automatische 明天-Briefing verbraucht ~1.500–3.000 Tokens 至 eingestellten 时间 punkt.

Wann ausschalten? & 阈值 e: Ausschalten, wenn kein täglicher 明天-Report gewünscht 是 (Std: 08:00 Uhr).

5. 多智能体流程模型、层级与综合 (Pillar 2)

43. 70 Deklarative 智能体 en-个人资料 e & Markdown-Personalakten

Problem: Generische KI-助手 en haben kein 角色 nbewusstsein 与 无 definierten 质量 sstandards.

Mechanik: Jeder 70 Spezialisten 是 在 agent_profiles.json sowie einer eigenen Personalakte (00_HR_智能体 en/) 与 System-Prompt, 模型 route, 工具-Scope 与 SOPs verankert.

收益: Schafft verlässliche virtuelle Fachkräfte 针对 战略, 法律, Bau, Finanzen, 营销 与 IT.

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

44. Robert Delegation, Fan-in-Synchronisation & Lisa Marker-Kontrakt (SYNTHESE-BEREIT)

Problem: Bei parallelen Teilaufgaben gingen Teilergebnisse verloren 或 wurden unvollständig zusammengefasst.

Mechanik: Robert zerlegt Großprojekte; ein 流程-Lock (_MC_DATA_LOCK) wartet auf den Abschluss aller

Teilaufgaben (Fan-in). Lisa aggregiert 报告 e 在…下 检验 des Pflicht-Markers SYNTHESE-BEREIT; Robert erteilt 在 Zeile 1 formale FREIGABE.

收益: *Mehrstufige 质量ssicherung 在…之后 dem Vorbild einer echten 企业sführung.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

45. 默认 isierter 智能体-to-Agent Handoff-Desk (backend/handoff_desk.py)

Problem: Stille Kontextverluste 在 Übergabe 的 einem Spezialisten an den nächsten.

Mechanik: Erzeugt formale Übergabeprotokolle 与 Deliverables, offenen Annahmen 与 SHA-256-Prüfsumme, vom Empfänger-智能体 en quittiert 将 müssen.

收益: *Revisionssichere 接口 n 之间 allen 部门 en.*

Preis & Schattenseite (Trade-off): Strikte SLA-检验 en weisen 很 kurze, aber korrekte Faktenantworten zurück, wenn min_content_length zu hoch (> 300) ist.

Wann ausschalten? & 阈值 e: 默认: 100 Zeichen Mindestlänge, 60 Min. Turnaround.

46. Zweistufiger DAG-Scheduler 针对 parallele 项目-执行 (backend/dag_scheduler.py)

Problem: Sequentielle Abarbeitung vieler Teilaufgaben dauert 在 Großprojekten zu lange.

Mechanik: Berechnet den gerichteten azyklischen Graphen (DAG): Führt unabhängige Recherchen (Stufe 1) parallel aus 与 startet danach 自动 Synthese (Stufe 2).

收益: *Verkürzt Durchlaufzeit komplexer 分析 n um ein Vielfaches.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

47. Self-Correction & Tool Execution Reflection Loop

Problem: Flüchtige Syntax- 或 参数 fehler 在 Tool-Aufrufen führten 至 sofortigen Abbruch.

Mechanik: Fängt Tool-错误 ab, reflektiert 错误 meldung 与 korrigiert Aufrufparameter autonom 在 bis zu 3 Versuchen.

收益: *Deutlich höhere Robustheit 在 文件-, 数据库- 与 Web-Operationen.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

48. Unterbrechbare Langzeit-工作流 s 与 SQLite State-Snapshotting

Problem: Wartet ein Task stundenlang auf eine Nutzer-批准, belegt er unnötig 存储 或 verliert seinen Zustand 在 einem 新建 start.

Mechanik: Friert den exakten 执行 szustand 在 表格 workflow_checkpoints ein 与 nimmt Arbeit 在…之后 批准 nahtlos wieder auf.

收益: *Ermöglicht mehrtägige Geschäftsprozesse 关于 新建 starts hinweg.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert

(vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

49. Dual-智能体 共识-Gate & Adversarial Critic Review (backend/consensus_gate.py)

Problem: Ein einzelner 智能体 bemerkt eigene Denkfehler 或 Halluzinationen 在 kritischen 任务 n oft nicht.

Mechanik: Erzwingt 在 P3-Tasks 与 kritischen Eingriffen ein formales Gegenlesen 通过 einen unabhängigen Revisor-智能体 en (Red-团队/Critic). Im Kanban-Board 作为 automatisiertes 4-Augen 质量 s-Gate verdrahtet (Katharina Völk / Robert Review 与 1–100% Score 与 状态-Lock 在 Revisionsbedarf).

收益: *Vier-Augen-Prinzip 针对 所有 geschäftskritischen 结果 se.*

Preis & Schattenseite (Trade-off): Verdreifacht (3x) den Token-Verbrauch 与 erhöht 回答 zeit um +6 bis +20 Sek.; Einstimmigkeit birgt Deadlock-Gefahr.

Wann ausschalten? & 阈值 e: Erst ab 优先级 P3/P4 aktivieren 与 2/3-Mehrheit wählen.

6. MCP-Treiber, 外部 e 服务 e & Multimodale 工具 e (Pillar 3)

50. Archon CRM & PocketBase-Ökosystem (Ports 3005, 8090, 8092)

Problem: 客户 nkontakte, 电子邮件-Historien 与 Medien-Assets lagen getrennt 的 den KI-智能体 en.

Mechanik: Dediziertes CRM-Frontend (Port 3005) 与 PocketBase-Backend (Port 8090) 与 Media-服务器 (Port 8092), voll angebunden 关于 scripts/archon_crm_mcp.py.

收益: *智能体 en 可 Kontakte abrufen, nachrecherchieren, bewerten (Lead-Scoring) 与 Korrespondenz zuordnen.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

51. 自动 es 电子邮件-Matching & 链接 edIn-Archiv-导入 er

Problem: Tausende Kontakte aus 链接 edIn-导出 en 与 电子邮件-Postfächern mussten 手动 abgeglichen werden.

Mechanik: Synchronisiert Postfächer (--fetch-mails), ordnet Mails 自动 bestehenden CRM-Kontakten zu 与 importiert komplette 链接 edIn-Connections-CSVs inklusive Deduplizierung.

收益: *Baut 在 Minuten eine vollständige, angereicherte Beziehungsdatenbank auf.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

52. Robert Tool-Permission Manager (backend/tool_permission_manager.py)

Problem: 智能体 en sollen 在 fehlenden Python-/Node-Paketen 不 hilflos abbrechen, dürfen aber 也 nichts ungefragt installieren.

Mechanik: Robert stellt einen formellen 安装 santrag (pending_permission) 与 Befehl 与 Begründung; erst 在…之后 deiner 1-Klick-批准 将 Paket installiert.

收益: *Kontrollierte Erweiterbarkeit 在 vollem 保护 des Host-Systems.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

53. Event-Driven Webhook-Mesh (backend/event_mesh.py)

Problem: 外部 e 事件 se aus GitHub, 表单 en 或 Drittsystemen konnten 无 智能体 en-Aktionen auslösen.

Mechanik: Universeller Webhook-Empfänger, externe JSON-Payloads validiert 与 direkt 在 ausführbare Kanban-Tasks umwandelt.

收益: *Verbindet Archon OS 在 Echtzeit 与 restlichen IT-Landschaft.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

54. Zentraler Multi-Channel Notification Hub (backend/notification_hub.py)

Problem: Wichtige 警告 en gingen 在 UI unter, während unwichtige Meldungen auf dem Handy störten.

Mechanik: Routet Meldungen strikt 在…之后 Schweregrad: INFO -> UI-Toast, WARNING -> 仪表板-Log, CRITICAL -> sofortiger Telegram-Push.

收益: *目标 genaue 告警 ierung 无 通知 s-Müdigkeit.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

55. Multi-Modale Vision- & Screen-Capture-Pipeline (backend/vision_pipeline.py)

Problem: Visuelle Probleme 或 表格 n 在 图像 ern mussten mühsam beschrieben werden.

Mechanik: Direkte 图像 schirmaufnahme 在 Browser (Kamera-Button 在 聊天) sowie automatische OCR- 与 Strukturanalyse 针对 hochgeladene 图像 er (.png, .jpg, .webp).

收益: *智能体 en sehen genau das, was du auf dem 图像 schirm siehst.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

56. Apollo Voice Engine — Edge-TTS & Whisper Diktat (apollo_voice_engine.py)

Problem: Lange 文本 e 在 Smartphone 或 Desktop einzutippen kostet 时间; 报告 e möchte man sich unterwegs anhören.

Mechanik: Bidirektionale Sprachpipeline: Mikrofon-Diktat (Speech-to-文本) direkt 在 聊天-输入 feld 与 natürliche Sprachausgabe (文本-to-Speech) 针对 Briefings. Vollständige Telegram-Voice-集成 (POST /api/telegram/voice-relay 与 POST /api/voice/synthesize) 针对 mobile 音频-Memos via edge-tts (de-DE-Conrad 新建 ral).

收益: *Verwandelt Archon OS 在 einen sprachgesteuerten Executive Assistant.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

57. Oracle News Radar & SEO Execution 智能体

Problem: 市场 veränderungen 与 SEO-漏洞 n eigener Webseiten wurden 仅 sporadisch geprüft.

Mechanik: Oracle News Radar scannt kontinuierlich Fachquellen auf 项目 relevanz; SEO 智能体 prüft 页 n auf Meta-Tags, Struktur 与 Keyword-Dichte.

收益: *持续e 市场- 与 Sichtbarkeitsoptimierung 在 Hintergrund.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

7. 记忆, 数据库-Hygiene & Obsidian 2nd Brain (Pillar 4)

58. Zentraler SQLite WAL-Memory-Layer (5 默认 tabellen)

Problem: Gleichzeitige Lese- 与 Schreibzugriffe mehrerer 智能体 en führten 在 einfachen 文件 en zu Konflikten.

Mechanik: Transaktionale SQLite-数据库 (data/agentic_memory.db) 在 Write-Ahead-Logging-Modus (WAL) 与 den 5 内核 tabellen tasks, conversations, learnings, kv_store 与 user_profile.

收益: *Blockadefreie Parallelzugriffe 与 höchster Datensicherheit.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

59. 流程 sicheres Kanban-Sidecar 与 atomarem 文件 austausch (backend/kanban_sidecar.py)

Problem: Metadaten 如 WIP-Limits 或 Tags konnten 在 parallelen Schreibvorgängen korrumpieren.

Mechanik: Schützt config/kanban_meta.json 关于 einen prozessweiten Mutex 与 atomare Write-Replace-Operationen.

收益: *Verhindert Datenverlust 在 hochfrequenten Board-更新 en.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

60. Docker-RAG-Bridge zu AnythingLLM (5.068 indexierte Dokumente auf Port 3001)

Problem: Das 在 Docker-Vektorspeicher vorhandene 知识 wurde 在 normalen 聊天-请求 n oft 不 自动 herangezogen.

Mechanik: backend/rag_bridge.py verbindet den 智能体 enkern direkt 与 dem AnythingLLM-Workspace ARCHON 与 liefert relevante 文本 passagen 作为 Grounded Context.

收益: *Fundierte 回答en auf Basis deiner echten 企业sdokumente statt allgemeiner Schätzungen.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

61. Dreistufige 项目 ordner-Disposition 在 Obsidian-Vault

Problem: Wenn 智能体 en 文件 en erzeugen, entsteht 无 feste Regeln schnell Unordnung 在 Vault.

Mechanik: Legt 针对 每 项目 自动 3-Stufen-Struktur an: 01_Zwischenberichte, 02_Medien 与 03_Endprodukte.

收益: *Saubere Trennung 之间 Rohentwürfen 与 abgenommenen 结束 rgebnissen.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

62. Obsidian Vault Indexer & 自动 e Map of Content (backend/vault_hygiene.py)

Problem: 新建 e 项目 berichte blieben 在 Obsidian-Graph oft unverlinkt ('Orphan Notes').

Mechanik: Scannt den Vault, spürt verwaiste Notizen auf 与 aktualisiert vollautomatisch zentrale _MOC_项目 _Index.md 与 Wikilinks.

收益: *Hält den 知识 sgraphen 在 Obsidian jederzeit vollständig vernetzt.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

63. Autonomes Fact-Harvesting & 85%-Ähnlichkeits-Deduplizierung (backend/post_task_pipeline.py)

Problem: Wichtige Erkenntnisse aus abgeschlossenen 任务 n gingen 在…下 或 wurden mehrfach doppelt gespeichert.

Mechanik: Extrahiert 在…之后 Task-Abschluss 自动 neue Fakten 针对 learnings 与 user_profile 与 verhindert Dubletten ab 85 % 文本 ähnlichkeit.

收益: *Das System 将 与 每 erledigten 任务 klüger, 无 den 存储 zuzumüllen.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

64. 自动 es Online-DB-备份 与 7-Tage-Rotation (backend/db_backup_manager.py)

Problem: Ohne regelmäßige 数据库 sicherungen droht 在 Hardwaredefekten 亏损 aller Tasks 与 个人资料 e.

Mechanik: Führt 在 laufenden 运行 PRAGMA wal_checkpoint(TRUNCATE) aus 与 speichert komprimierte .db.gz-Archive 在 .hermes/backups/db/.

收益: *Lückenlose Datensicherung 无 eine Sekunde Ausfallzeit.*

Preis & Schattenseite (Trade-off): 每日 e Voll-备份 s belegen je 在…之后 归档 sdauer ~50–500 MB 磁盘 nspeicher.

Wann ausschalten? & 阈值 e: 默认: 7 Tage 归档 (db_backup_keep_days).

65. Autonomer SQLite Vacuum & Fragmentierungs-Optimizer (backend/db_optimizer.py)

Problem: Gelöschte Datensätze 与 häufige 更新 s blähen SQLite-文件 en 关于 Monate unnötig auf.

Mechanik: Führt 关于 POST /api/system/optimize-db auf allen 数据库 en PRAGMA optimize 与 VACUUM aus 与 berechnet den freigegebenen 存储 platz.

收益: *Hält 查询 zeiten dauerhaft 在 Millisekundenbereich.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert

(vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

66. Atomares Pre-Write-Snapshotting & 1-Klick-Rollback (backend/snapshot_manager.py)

Problem: Wenn ein 智能体 eine 文件 fehlerhaft überschreibt, war 恢复 des Vorzustands aufwendig.

Mechanik: Sichert 在…之前 jedem Schreibzugriff 自动 Original 在 .hermes/snapshots/ 与 stellt es per Klick 或 API sofort wieder her. Zusätzlich 作为 Pre-更新 Snapshots aller SQLite-数据库 en 在…之前 GitHub-Pulls 与 容器-更新 s 在 data/snapshots/ integriert.

收益: *Vollständiges 安全 snetz 在 allen 文件- 与 Code-变更 en.*

Preis & Schattenseite (Trade-off): Jeder Pre-Write-Snapshot kostet ~5–15 ms Disk-I/O 在…之前 dem Überschreiben einer 文件.

Wann ausschalten? & 阈值 e: Ringpuffer standardmäßig auf 50 Snapshots begrenzt.

67. Git-kompatible Live-Diff-Vorschau (backend/diff_engine.py)

Problem: Vor einer 批准 möchte man exakt sehen, welche Zeilen sich ändern, statt ganze 文件 新 zu lesen.

Mechanik: Erzeugt einen farbigen Zeilenvergleich (Rot = entfernt, Grün = neu) 之间 库存 与 智能体 en-设计.

收益: *Sekundenschnelle 检验 的 Code- 与 Dokumentenänderungen 在 HITL-Gate.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

8. 安全、零信任治理与系统诊断 (Pillar 6)

68. Kryptografisch verkettetes SHA-256 审计-Log (backend/audit_log.py)

Problem: 简单 e 文本-Logs lassen sich nachträglich verändern 或 löschen.

Mechanik: Jedes 事件 将 与 dem SHA-256-Hash des Vorgängereintrags verkettet (Blockchain-Prinzip) 与 是 关于 GET /api/audit/verify auf Manipulation prüfbar.

收益: *Revisionssicherer 证明 aller System- 与 批准 aktionen.*

Preis & Schattenseite (Trade-off): Minimaler Schreib-Overhead (~2 ms pro Eintrag); kryptografische Kette wächst 关于 Monate an.

Wann ausschalten? & 阈值 e: Für revisionssichere Governance dauerhaft eingeschaltet lassen.

69. Local Security Guard — 25 Prompt-Injection- & Jailbreak-Signaturen

Problem: Bösartige 电子邮件 s 或 Webseiten 可 versteckte Befehle enthalten ('Ignore previous instructions...').

Mechanik: Scannt jeden externen 文本 eingang anhand 的 25 攻击 smustern, bevor er 在 den Kontext eines 智能体 en gelangt.

收益: *Wehrt indirekte Prompt-Injections an Systemgrenze ab.*

Preis & Schattenseite (Trade-off): Sehr strenge Heuristiken 可 在 IT-Security-文本 en 或 Pen-测试 ing-Prompts Fehlalarme (False Positives) auslösen.

Wann ausschalten? & 阈值 e: Nur 针对 autorisierte Red-团队 ing-测试 s kurzzeitig lockern.

70. Secret-Masking & Red-团队 ing Output Guard (backend/secret_masker.py)

Problem: API-密钥 或 Passwörter könnten versehentlich 在 Logs, 报告 en 或 聊天-输出 n auftauchen.

Mechanik: 筛选 t ausgehende 文本 e 在 Echtzeit auf 密钥-Muster (sk-..., Bearer, Passwörter) 与 ersetzt sie 通过 [GESCHÜTZTER_KEY].

收益: *Verhindert versehentliche Offenlegen vertraulicher Zugangsdaten.*

Preis & Schattenseite (Trade-off): Aggressive Regex-Maskierung 可 在 seltenen Fällen harmlose Base64- 或 Hash-Strings 在 Code-输出 n schwärzen.

Wann ausschalten? & 阈值 e: Im Produktivbetrieb 至 保护 的 API-Keys dauerhaft aktiv lassen.

71. Abgesicherter Docker-Socket-代理 (Port 2375 intern)

Problem: Direkter 访问 auf /var/run/docker.sock gibt 容器 n uneingeschränkte Root-法律 e 关于 Host-System.

Mechanik: Schaltet einen restriktiven 代理-容器 dazwischen, 仅 explizit erlaubte 状态- 与 新建 start-Befehle durchlässt.

收益: *Schützt Host-运行 ssystem 在…之后 dem Zero-Trust-Prinzip.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

72. A/B-Modell-Benchmarker & Empirischer Performance-Scorer (backend/model_benchmarker.py)

Problem: 新建 e Sprachmodelle erscheinen wöchentlich; ihre Eignung 针对 eigene 企业 saufgaben bleibt oft unklar.

Mechanik: 测试 et zwei 模型 e parallel 与 identischen 任务 n 与 vergleicht 回答 zeit, Token-成本 与 结果 qualität.

收益: *Objektive Datenbasis 针对 模型 wahl 在 智能体团队.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

73. Systemweiter 360°-Health-Score & Diagnose-Engine (backend/system_diagnostics.py)

Problem: Einzelne 指标 en zeigen oft 不 auf einen Blick, 如 gesund Gesamtsystem wirklich ist.

Mechanik: Berechnet aus Docker-Flottenstatus (35%), Concurrency-Queue (20%), 审计-完整性 (20%), Deadlock-Freiheit (15%) 与 RAG-Erreichbarkeit (10%) einen Gesamtscore 的 0–100 % inkl. Handlungsempfehlungen.

收益: *立即 ige Gesamtdiagnose des 运行 ssystems 关于 GET /api/system/health-score.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

9. 自进化、市场与自动文档 (Pillar 7)

74. Autonome Skill-Synthese 与 Personalakten-Rückkopplung (backend/skill_synthesizer.py)

Problem: Erfolgreiche mehrstufige Lösungswege mussten bisher 手动 作为 neuer Skill programmiert werden.

Mechanik: Erkennt wiederkehrende erfolgreiche Tool-Ketten, generiert 自动 einen SKILL.md-设计, trägt ihn 在 Personalakte des 智能体 en ein 与 legt ihn 至 批准 vor.

收益: *Das System erweitert seine 能力 en 在 运行 selbstständig.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

75. Dynamischer Talent- & Skill-市场 platz (skill_markt.js & 智能体 SkillsHub)

Problem: 新建 e 角色 n 或 工作流 s 针对 neue Geschäftsfelder 的 Grund auf 新 zu schreiben, 是 zeitintensiv.

Mechanik: Durchsuchbarer Katalog 至 1-Klick-导入 与 -导出 kompletter 智能体 en-个人资料 e sowie externer Skills (inkl. agentskillshub.dev 集成).

收益: *Skaliert 智能体 en-Flotte 在 neuen 项目 en 在 Sekunden.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

76. Vollständige Frontend-Backend-Synchronisation (238 Fetches / 581 路径 e / 29 Modale)

Problem: In großen Web-应用 en führen verwaiste Buttons 或 unverbundene Modale oft zu stillen 错误 n.

Mechanik: Automatisierte Prüf-Pipeline stellt sicher, dass 所有 238 Frontend-API-Aufrufe 与 29 Modale exakt 与 den 368 Backend-Routen verdrahtet sind.

收益: *担保 rt 0 tote 链接 s 与 100 % bedienbare 税 elemente 在 allen 视图 en.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

77. Autonomer Master-Dossier- & 手册-Generator (backend/system_dossier_generator.py)

Problem: 架构-文档 en veralten meist 已 wenige Tage 在…之后 ihrer Erstellung.

Mechanik: Generiert auf Knopfdruck (POST /api/system/generate-dossier) aktuelle System-架构-Dossier simultan 在 项目 ordner 与 在 Obsidian-Vault (00_DOKUMENTATION/).

收益: *Hält Systemdokumentation 无 manuellen Pflegeaufwand dauerhaft auf dem neuesten Stand.*

Preis & Schattenseite (Trade-off): Als permanente 内核-视图 bzw. System-基础设施 在 仪表板 integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-成本 entstehen 仅 在 aktiver 执行 的 LLM-Aktionen 之内 des 模块 s).

10. Referenz-Matrix: 税 bare Evey/Hermes-Plugins

Nr. & 插件-ID	Fach-类别	内核 mechanik & 功能 在 运行 ssystem
78. evey-cost-guard	安全 & Wächter	Echtzeit-成本 überwachung 与 harte 预算-Abschaltung

		pro Tag 与 Task.
79. evey-session-guard	安全 & Wächter	Überwacht 会话 slaufzeiten, Token-Obergrenzen 与 verhindert Zombie-Sessions.
80. evey-email-guard	安全 & Wächter	Scannt eingehende 电子邮件 s 在…之前 Triage auf Phishing 与 Prompt-Injections.
81. evey-watchdog	安全 & Wächter	Prüft 在 30s-Takt aktive 流程 e 与 startet blockierte Worker 自动 neu.
82. evey-cache	Performance & 缓存	24h-回答-缓存 至 sofortigen, kostenfreien Beantwortung wiederholter 查询 n.
83. disk-cleanup	Performance & 维护	Bereinigt temporäre Artefakte, veraltete Logs 与 gibt 磁盘 nspeicher frei.
84. evey-telemetry	Monitoring & Analytics	Erfasst Token-Ströme, 延迟 en 与 模型-Aufrufe 针对 Mission Control 与 Guardian.
85. evey-status	Monitoring & Analytics	持续 er Health-Ping auf 所有 11 Docker-容器 与 网关 s.
86. evey-delegation-score	Monitoring & Analytics	Bewertet Erfolgs- 与 质量 squote 的 任务 nübergaben 之间 智能体 en.
87. evey-autonomy	智能体 en-Intelligenz	税 t den Autonomiegrad 在 mehrstufigen Hintergrund-决策 en.
88. evey-council	智能体 en-Intelligenz	多数 s- 与 共识 bildung 之间 mehreren Spezialisten (Mastermind-Kopplung).
89. evey-reflect	智能体 en-Intelligenz	Selbstreflexions-Schleife 至 eigenständigen 错误 korrektur 在…之前 结果 abgabe.
90. evey-learner	智能体 en-Intelligenz	Extrahiert dauerhafte Erkenntnisse aus erledigten 任务 n 针对 learnings-表格.
91. evey-identity	智能体 en-Intelligenz	Sichert strikte Einhaltung 的 SOUL.md, Tonalität 与 角色 n-Leitplanken.
92. evey-habits	智能体 en-Intelligenz	Verwaltet wiederkehrende Routinen 如 tägliche 08:00-Uhr-Standup-Briefing.
93. evey-proactive	智能体 en-Intelligenz	Erlaubt 智能体 en, 在 erkannten Engpässen selbstständig Handlungsvorschläge zu machen.
94. evey-goals	规划 & Tasks	Verknüpft operative Einzelaufgaben 在 Kanban 与 übergeordneten 企业 szielen.
95. evey-scheduler	规划 & Tasks	时间- 与 prioritätsgesteuerte Einplanung 的 Hintergrund-

		Aufträgen.
96. evey-bridge	Brücken & 接口 n	内部 e Datendrehscheibe 之间 FastAPI-Monolith, Hermes-守护进程 与 Sidecars.
97. evey-mqtt	Brücken & 接口 n	Leichtgewichtiges 消息 enprotokoll 针对 IoT- 与 Echtzeit-状态 signale.
98. evey-moltbook	Brücken & 接口 n	Synchronisiert strukturierte Arbeitsbücher 与 Synthese-Notizen.
99. evey-telegram-ux	Brücken & 接口 n	税 t Rufnamen-Routing (/robert, /anwalt) 与 interaktive 批准-Buttons 在 Telegram.
100. evey-github	Brücken & 接口 n	Anbindung 针对 Repository-Operationen, Commit-分析 n 与 Issue-Tracking.
101. evey-research	Tools & Recherche	Tiefensuche-Treiber 针对 Web-Scraping 与 来源 n-Extraktion 在 Research Lab.
102. evey-news	Tools & Recherche	Hintergrund-Scanner 针对 Branchen-Feeds 与 市场-新建igkeiten (Oracle News Radar).
103. evey-digest	Tools & Recherche	Komprimiert umfangreiche Dokumentensammlungen 在 kompakte Management-Summaries.
104. evey-sandbox	Tools & Execution	Isolierte 执行 sumgebung 针对 generierten Python- 与 分析-Code 在 Showroom.
105. evey-rag	记忆 & 知识	Vektor-Retrieval-Treiber 至 Anbindung 的 AnythingLLM 与 lokalen 知识 sarchiven.
106. evey-memory-adaptive	记忆 & 知识	Dynamische Gewichtung 的 Kurzzeit- 与 Langzeitgedächtnis je 在…之后 Kontextdruck.
107. evey-memory-consolidate	记忆 & 知识	Nachtlauf 至 Zusammenführung 与 Deduplizierung verwandter 记忆 einträge.
108. evey-wallet	Finanzen & 预算 s	Verwaltet API-Guthaben, Kontingente 与 kostenbezogene Transaktions-Limits.
109. evey-validate	质量 & 执行	Prüft 输出 n auf Schema-Konformität, Pflicht-Marker (SYNTHESE-BEREIT) 与 Formatvorgaben.
110. evey-verification	质量 & 执行	Faktencheck 与 Gegenprüfung kritischer Aussagen 在…之前 Endfreigabe.
111. evey-delegate-model	质量 & 执行	Wählt 在 Teilaufgaben 自动 kosten- 与 leistungsoptimale 目标 modell.

11. 企业与弹性架构

78. Replay & Time-Travel Debugging (backend/time_travel_debugger.py)

Problem: Bei komplexen 智能体 en-工作流 s 与 vielen Tool-Calls 是 nachträgliche 错误 diagnose 在 文本 log unübersichtlich 与 Verzweigen 的 执行 sträng en unmöglich.

Mechanik: 存储 t jeden Teilschritt 与 Ein-/Ausgaben, Snapshots 与 Tokenverbrauch 在 'execution_traces'. Ermöglicht interaktives Replay (GET /api/traces/{task_id}), State-Forking ab 步骤 N (POST /api/traces/{task_id}/fork) 与 1-Klick-Rollback modifizierter 文件 en.

收益: 快速 e Ursachenanalyse 与 risikofreie Was-wäre-wenn-Experimente 与 alternativen Prompts 或 模型 en 无 新建 start gesamten Pipeline.

Preis & Schattenseite (Trade-off): Erhöht Disk-I/O um ~5–15 ms pro 文件-Edit 与 belegt je 在…之后 Snapshot-Limit ~50–250 MB zusätzlichen 存储 platz auf 磁盘.

Wann ausschalten? & 阈值 e: Ausschalten auf Systemen 与 extrem knappem Disk-存储 或 在 massenhaften 文件-Generierungen (z. B. Build-Artefakte). | 阈值 e: Max Snapshots: < 10 = kurze Historie | > 200 = höherer Disk-Verbrauch (Std: 50). Max Trace-Steps: Std 100.

79. Multi-模型 Eval-Matrix & A/B-Benchmarker (backend/model_benchmarker.py)

Problem: 模型-Zuweisungen an 智能体 en basieren mangels empirischer Vergleichswerte oft auf subjektiven Annahmen.

Mechanik: Sendet denselben Prompt simultan an zwei konfigurierte LLMs (POST /api/router/benchmark), misst 延迟, Token-Verbrauch 与 回答 länge 与 speichert Historie persistent 在 'model_benchmarks' (GET /api/router/benchmark/history).

收益: Objektive, datengestützte 模型 auswahl je 智能体 enrolle (z. B. DeepSeek V3 针对 快速 igkeit/Kosten vs. Claude 3.7 针对 komplexe 架构 en).

Preis & Schattenseite (Trade-off): N Kandidaten-模型 e kosten N-fache API-Tokens pro 测试 lauf.

Wann ausschalten? & 阈值 e: Wann ausschalten? Nur gezielt 针对 模型 vergleiche starten, 不 在 持续时间 betrieb.

80. Tri-智能体 共识-Gate & Schiedsgericht (backend/consensus_gate.py)

Problem: 严重 e Eingriffe (Code-Refactorings, 迁移 en) bergen 在 Einzelentscheidungen 风险 的 Halluzinationen 或 安全 slücken.

Mechanik: Beruft ein dreiköpfiges Gremium (System Architect, Security Officer, QA Lead) ein (POST /api/consensus/council-vote). Jeder Revisor votiert unabhängig; 批准 erfordert 2/3-Mehrheit. Alle Beschlüsse 将 在 'consensus_decisions' archiviert (GET /api/consensus/history).

收益: Verlässliche 风险 minimierung 与 Eliminierung 的 Single-Point-of-Failure-Halluzinationen 在 sensiblen Systemänderungen.

Preis & Schattenseite (Trade-off): Verdreifacht (3x) den Token-Verbrauch 与 erhöht 回答 zeit um +6 bis +20 Sek. Bei Einstimmigkeits-Zwang droht 决策 s-Stillstand (Deadlock).

Wann ausschalten? & 阈值 e: Ausschalten (oder erst ab 优先级 4 = 严重 aktivieren), wenn Geschwindigkeit 与 niedrige API-成本 Vorrang haben. | 阈值 e: Ab 优先级: Prio 2 = fast 每 Task kostet 3x Tokens | Prio 4 = 仅 kritische Tasks (Std: Prio 3). 多数: 2/3-Mehrheit empfohlen.

81. Autonome Task-Selbstheilung 与 Git-Rollback (backend/task_self_healer.py)

Problem: Syntaxfehler 或 fehlerhafte Code-导入 s blockieren Tasks sofort 与 erfordern manuelles Eingreifen des

Nutzers.

Mechanik: Isoliert den Stacktrace, führt 在 期限 fehlern einen Git-Rollback modifizierter 文件 en 通过 与 übergibt dem 智能体 en einen strukturierten Korrektur-Prompt (POST /api/healing/attempt). Bis zu 3 autonome Reparaturversuche; Log-Abruf via GET /api/healing/logs.

收益: Maximale 韧性 在 Code- 与 Skript-执行 en 与 automatischer Selbstreparatur 在…之前 升级 ins HITL-Board.

Preis & Schattenseite (Trade-off): Jeder Selbstheilungsversuch kostet 1 zusätzlichen LLM-Call (+3 bis +12 Sek. 延迟 & Token-成本). Ein automatisches Git-Rollback verwirft 在 Fehlschlag 也 nützliche Teilerfolge.

Wann ausschalten? & 阈值 e: Git-Rollback ausschalten, wenn unfertige Code-Entwürfe 至 manuellen 下一步 arbeit 在 Arbeitsverzeichnis stehen bleiben sollen. | 阈值 e: Max Reparaturversuche: 1 = gibt sofort auf | > 5 = teure Reparatur-Kaskade (Std: 3).

82. Event-Mesh & Posteingangs-目录-Watcher (backend/event_mesh.py)

Problem: 任务 n mussten bisher 手动 关于 仪表板 angestoßen 或 关于 zeitgesteuertes IMAP-Polling abgefragt werden.

Mechanik: Zentraler Inbound-Event-Bus 针对 externe Webhooks (POST /api/events/inbound/{channel}) 与 automatischer 目录-Watcher (POST /api/events/scan-directory), neue 文件 en 在 'data/inbox/' direkt 在 Kanban-Triage-Tasks überführt.

收益: Vollautomatische Event- 与 Belegerfassung aus externen Systemen (GitHub, n8n, Stripe, lokale 文件 scanner) 无 用户 interaktion.

Preis & Schattenseite (Trade-off): Sehr kurze Scan-Intervalle (< 3 Sek.) erzeugen kontinuierliche 文件 system-Polls; tägliche 明天-Standup verbraucht ~1.500–3.000 Tokens pro Lauf.

Wann ausschalten? & 阈值 e: Ausschalten, wenn lokale Inbox-文件夹 不 aktiv genutzt 将 或 kein morgendlicher Briefing-报告 gewünscht ist. | 阈值 e: Scan-Intervall: 1–2s = unnötige Disk-Polls | > 30s = verzögerte 文件-Erkennung (Std: 5s). Standup-时间: Std 08:00 Uhr.

12. 自主、沙箱与治理架构

83. Flüchtige Code-沙箱 en (backend/ephemeral_sandbox.py)

Problem: 执行 的 智能体 en-Code birgt 风险 的 Systemabstürzen, Endlosschleifen 或 unbeabsichtigten 文件 änderungen auf dem Host.

Mechanik: Führt Code 在 kurzlebigen, isolierten Docker-Micro-容器 n 与 harter 资源 n-Begrenzung (512 MB RAM, 1 CPU, 30s Timeout, kein Netz) aus (POST /api/sandbox/execute). Die 沙箱 将 在…之后 Abschluss rückstandslos gelöscht.

收益: Absolute 安全 与 Systemstabilität 在 automatisierten Code-执行 与 -Erprobung.

Preis & Schattenseite (Trade-off): 容器-Kaltstart kostet ~300–800 ms Zusatzlatenz pro Tool-Call. Ist 网络 deaktiviert, schlagen pip install 或 externe API-Abrufe 在 Skript sofort fehl.

Wann ausschalten? & 阈值 e: Ausschalten (oder 网络 aktivieren), wenn vertrauenswürdige lokale Skripte direkten Host-访问 或 schnelle Paketinstallationen benötigen. | 阈值 e: RAM-Limit: < 256 MB = OOM-Kill 在 Pandas/NumPy | > 2048 MB = Host-RAM-风险 (Std: 512 MB). Timeout: < 10s = bricht Builds ab (Std: 30s).

84. Nächtliche 记忆-Konsolidierung (backend/memory_consolidator.py)

Problem: Stetig anwachsende Traces, Heartbeats 与 veraltete 对话 daten verlangsamen 数据库 abfragen 在 持续时间

betrieb.

Mechanik: Prunt 在 Ruhephasen (默认 03:00 Uhr) veraltete Traces 与 unkritische Logs anhand konfigurierbarer Retention-期限 en (settings.json). Vollständig steuerbar 关于 Hermes Core 设置 en (Uhrzeit, Retention-Tage, Pruning-Tiefe, manueller Run-Button).

收益: 持续时间 haft schlanke 数据库 en, reduzierte 延迟 en 与 automatische System-Hygiene 无 manuellen 维护 saufwand.

Preis & Schattenseite (Trade-off): Während des Konsolidierungslaufs (~10–30 Sek.) entsteht kurze CPU-/Disk-Last; eine zu niedrige Pruning-Schwelle (< 0.88) 可 verwandte, aber eigenständige Fakten zusammenfassen.

Wann ausschalten? & 阈值 e: Ausschalten, wenn lückenlose Roh-Traces 针对 ein forensisches 审计 dauerhaft ungefiltert erhalten bleiben müssen. | 阈值 e: 归档: < 7 Tage = 亏损 wichtiger 审计-Spuren | > 60 Tage = große SQLite-DB (Std: 14 Tage). Pruning-Schwelle: 0.95 = konservativ | 0.88 = aggressiv (Std: 0.92).

85. Visueller Web-智能体 与 Screenshot-审计 (backend/visual_web_agent.py)

Problem: 任务 n auf komplexen Webportalen 无 API (SPAs, 表单 e, interaktive 仪表板 s) 可 与 文本-Scrapern 不 bedient werden.

Mechanik: 税 t einen echten Headless-Chromium-Browser (POST /api/browser/browse-and-capture), erfasst Screenshots 针对 jeden Klick- 与 输入 schritt 在 .hermes/visual_traces/{task_id}/ 与 extrahiert strukturierte Daten.

收益: Lückenlose visuelle Nachvollziehbarkeit 与 automatisierte Interaktion 与 modernen Webanwendungen 无 API-Zugang.

Preis & Schattenseite (Trade-off): Headless-Chromium benötigt pro aktivem 审计-Lauf ~300 MB RAM 与 ~3–5 Sek. Renderzeit.

Wann ausschalten? & 阈值 e: Wann ausschalten? Bei reinem Backend-运行 无 UI-检验 en.

86. Virtueller War-Room & Pre-Mortem Stresstest (backend/war_room_simulator.py)

Problem: Unvorhergesehene 安全 s-, Akzeptanz- 或 合规-Risiken 将 在 Produkteinführungen oft erst 在…之后 dem 推广 bemerkt.

Mechanik: Simuliert drei gegnerische Parteien (严重 er Großkunde, Offensiver Pentester, Strenger 合规-审计 or) 在…之前 批准 (POST /api/simulation/war-room). Erstellt eine aggregierte 风险-Matrix (LOW bis CRITICAL) 与 verbindlichen Auflagen.

收益: Proaktives Aufdecken 的 漏洞 n 与 Vermeidung kostspieliger Fehlentscheidungen 在…之前 dem Livegang.

Preis & Schattenseite (Trade-off): Kostet ~1.500–3.000 zusätzliche Tokens 与 +4 bis +10 Sek. Wartezeit 在…之前 dem eigentlichen Task. Bei Block-Schwelle MEDIUM entstehen häufiger Fehlalarme.

Wann ausschalten? & 阈值 e: Auto-Pre-Mortem 在 normalen 开发 salltag ausschalten 与 仅 针对 produktionskritische Eingriffe (HIGH/CRITICAL) nutzen. | 阈值 e: Block-Schwelle: MEDIUM = blockiert oft vorsorglich | CRITICAL = stoppt 仅 katastrophale Risiken (Std: HIGH).

87. SLA- & Übergabe-Verträge 之间 智能体 en (backend/sla_contract_enforcer.py)

Problem: Lose Delegationen 在 Multi-智能体 en-Hierarchien führen oft zu unvollständigen Zuarbeiten, an den menschlichen Operator eskaliert werden.

Mechanik: Schließt 在 internen Delegationen digitale Verträge 与 Pflichtfeldern, Mindestlänge 与 Akzeptanzkriterien ab (POST /api/contracts/create). Validiert 交付 en autonom (POST /api/contracts/validate) 与 weist mangelhafte 结果 se 自动 至 Nachbesserung ab.

收益: Strikte Einhaltung 的 质量 sstandards 在 智能体 en-团队 与 Vermeidung unfertiger Zwischenstufen 在 HITL-

Board.

Preis & Schattenseite (Trade-off): Wird Mindestlänge zu hoch angesetzt (> 300 Zeichen), 将 präzise Kurzantworten abgelehnt 与 lösen teure 返工 s-Schleifen aus.

Wann ausschalten? & 阈值 e: Auto-Reject ausschalten, wenn Subagenten 也 knappe 状态- 或 Ja/Nein-Antworten zurückgeben dürfen. | 阈值 e: Min. Länge: < 30 = lässt Platzhalter 通过 | > 300 = lehnt knappe Faktenantworten ab (Std: 100 Zeichen). Max Turnaround: Std 60 Min.

88. Zentrale Autonomie- & 阈值-控制 (backend/autonomy_settings.py)

Problem: Feste 阈值 e 在 Programmcode verhindern eine schnelle Anpassung 的 安全 s-, 沙箱- 与 韧性-Regeln 在 laufenden 运行.

Mechanik: Bündelt 所有 10 Autonomie-组 n (Concurrency, Loop-Killer, Selbstheilung, Docker-沙箱, Time-Travel-Snapshots, 共识-Gate, War-Room, SLA-Verträge, Inbox/Standup 与 Flotten-Hygiene) 在 settings.json 与 synchronisiert sie 关于 GET/POST /api/settings/autonomy-modules 与 dem System Control Center.

收益: Jedes 保护- 与 Autonomie-模块 lässt sich 至 期限 per Klick ein- 或 ausschalten 与 exakt auf gewünschte 风险- 与 资源 nprofil feinjustieren.

Preis & Schattenseite (Trade-off): Ein defekter 容器 可 通过 Auto-Restart 在 eine Crash-Loop geraten; visuelle Browser-审计 startet Chromium (+300 MB RAM, +4 Sek. 持续时间).

Wann ausschalten? & 阈值 e: Auto-Restart während manueller 容器-维护 ausschalten; Browser-审计 在 reinem Backend-运行 deaktivieren. | 阈值 e: DB-备份-Tage: < 3 Tage = knappe Historie | > 30 Tage = ~500 MB+ 备份-存储 (Std: 7 Tage).

13. 性能核心、吞吐扩展与 Token 效率

89. Semantisches Prompt- & Response-Caching (backend/semantic_cache.py)

Problem: Wiederkehrende 或 leicht umformulierte 智能体 en-请求 n verursachen unnötige LLM-API-成本 与 sekundenlange Wartezeiten.

Mechanik: Berechnet 针对 jeden Prompt einen SHA-256-Hash sowie einen L2-normalisierten Wort-/3-Gramm-Vektor 在 SQLite 与 liefert 在 einer Cosine-Ähnlichkeit >= 0.95 sofort den gespeicherten Response-Treffer.

收益: 回答 zeiten 在…下 10 Millisekunden 与 100 % Token-Ersparnis 在 semantisch identischen 请求 n.

Preis & Schattenseite (Trade-off): Ist Cosine-Schwelle zu niedrig (< 0.92), übersieht 缓存 feine Bedeutungsunterschiede (z. B. 'lösche 测试-DB' vs. 'lösche Prod-DB') 与 liefert falsche 或 veraltete 回答 en.

Wann ausschalten? & 阈值 e: Ausschalten 在 任务 n 与 Echtzeit-Daten (Börsenkurse, Live-Logs, Systemstatus) 或 wenn 每 回答 frisch generiert 将 muss. | 阈值 e: Cosine-Schwelle: < 0.92 = falsche 回答 en aus 缓存! | > 0.98 = kaum Treffer (Std: 0.95). TTL: Std 24h. Max Einträge: Std 500.

90. Paralleles Map-Reduce Sub-Tasking / Fan-Out & Fan-In (backend/map_reduce_executor.py)

Problem: Sequenzielle Abarbeitung vieler Einzeldokumente 或 Teilfragen führt zu linearem 时间 wachstum 与 langen Wartezeiten.

Mechanik: Teilt Listen-Aufträge 自动 auf bis zu N parallele Asyncio-Worker (Semaphore-gesteuert) auf 与 führt Teilergebnisse 在 Reduce-步骤 zusammen.

收益: Vervielfacht den 吞吐量 在 Dokumenten-, Recherche- 与 Code-Batch-分析 n 无 Überlastung Event-Loop.

Preis & Schattenseite (Trade-off): Erzeugt N-fache gleichzeitige API-Last (Gefahr 的 HTTP-429 Rate-Limits 在 Provider) 与 ~20–35 % Token-Overhead 通过 anschließende Reduce-Synthese.

Wann ausschalten? & 阈值 e: Ausschalten 在 strikten API-Rate-Limits 或 wenn Teilaufgaben eng voneinander abhängen 与 sequenziell aufeinander aufbauen müssen. | 阈值 e: Max Worker: > 8 = Rate-Limit-Gefahr (Std: 4). Chunk-Timeout: Std 30s. Ab Elementen: Std 3.

91. Speculative Fast-Routing & 延迟-Hedging (backend/speculative_hedger.py)

Problem: Einzelne langsame Cloud-Requests (Tail-Latency-Spikes) blockieren gesamte 智能体 en-执行.

Mechanik: 开始 et 在…之后 Ablauf 的 hedge_delay_seconds (z. B. 2.5s) 自动 einen parallelen Ersatz-Call auf einem schnellen Fallback-模型 与 übernimmt zuerst eintreffende 结果.

收益: Eliminiert Hänger 通过 langsame API-Endpunkte 与 stabilisiert 回答 zeiten 在 99. Perzentil.

Preis & Schattenseite (Trade-off): Wenn Primärmodell kurz 在…之后 dem Hedge-开始 doch 仍 antwortet, 将 beide Requests abgerechnet (bis zu 2x Token-成本 在 langsamen Prompts).

Wann ausschalten? & 阈值 e: Ausschalten 在 knappem API-预算 或 wenn langsame Reasoning-模型 e genutzt werden, regulär > 10 Sek. Denkzeit benötigen. | 阈值 e: Hedge-Verzögerung: < 1.5s = fast immer doppelte API-成本! | > 8s = kaum 延迟-保护 (Std: 2.5s).

92. Dynamisches Context-Compaction & Hierarchisches Summarizing (backend/context_compactor.py)

Problem: Lange 智能体 en-会话 en blähen Kontextfenster auf, verlangsamen den Prompt-Prefill 与 erhöhen 成本 pro Turn drastisch.

Mechanik: Extrahiert ab einem einstellbaren Token-阈值 所有 harten Fakten (目标 e, 文件 pfade, 决策 en, 错误) 在 einen kompakten [HIERARCHICAL_MEMORY_STATE]-Block 与 behält 仅 jüngsten Runden 在 Volltext.

收益: Spart 65–80 % Kontext-Tokens 在 langen Sessions 无 亏损 wichtiger 项目 daten.

Preis & Schattenseite (Trade-off): Der Komprimierungslauf kostet einmalig ~1.000 Tokens; exakte Code-Zeilen 或 kleine 详情 werte aus weit zurückliegenden 消息 en 可 在 总结 verloren gehen.

Wann ausschalten? & 阈值 e: Ausschalten, wenn 智能体 wortwörtlich auf 很 frühe 消息 en derselben 会话 zurückgreifen muss. | 阈值 e: Token-Schwelle: < 3000 = vergisst 详情 zu früh | > 24000 = hohe Prompt-成本 (Std: 6000). Letzte Turns behalten: Std 4.

93. SQLite WAL Connection-Pool & Async Write-Batching (backend/async_write_batcher.py)

Problem: Hunderte einzelne SQLite-INSERTs 在 parallelen 智能体 en-Läufen erzeugen Disk-I/O-Spitzen 与 'database is locked'-Konflikte.

Mechanik: Sammelt Telemetrie- 与 审计-事件 se 在 einem Thread-sicheren RAM-Ringpuffer 与 schreibt sie gebündelt via executemany() 在 WAL-Modus.

收益: Reduziert 磁盘 n-Schreibzugriffe um rund 80 % 与 hält Haupt-Event-Loop frei 的 Blockaden.

Preis & Schattenseite (Trade-off): Im Falle eines harten Stromausfalls 或 kill -9 在…之前 dem nächsten Flush-Intervall gehen 在 RAM gepufferten Einträge letzten 1–5 Sekunden verloren.

Wann ausschalten? & 阈值 e: Ausschalten, wenn 每 einzelne Log-Eintrag sofort 与 无 Sekundenverzögerung synchron auf 磁盘 liegen muss. | 阈值 e: Batch-Größe: < 5 = viele Disk-Writes | > 100 = 更多 亏损 在 Crash (Std: 25). Flush-Intervall: Std 2.0s.

14. Moderne 记忆- & RAG-架构

94. Hybrid-Search 与 BM25 FTS5 & Reciprocal Rank Fusion (backend/hybrid_search_engine.py)

Problem: Reine Vektorsuche findet semantisch ähnliche 文本 e, scheitert jedoch oft an exakten 错误 codes, 功能 snamen 或 Seriennummern.

Mechanik: Kombiniert SQLite FTS5 Volltext-Ranking (BM25) 与 Cosine-Vektor-Ähnlichkeit 关于 Reciprocal Rank Fusion (RRF).

收益: Vereint exakte Keyword-Treffer 与 semantisches Verständnis 在 einer einzigen, hochpräzisen Trefferliste.

Preis & Schattenseite (Trade-off): Doppelte Index-查询 erhöht Such-延迟 um ~15–40 ms; ein zu hohes Top-K (> 10) füllt den 智能体 en-Prompt 与 tausenden Zusatz-Tokens.

Wann ausschalten? & 阈值 e: Fast nie ausschalten — höchstens Top-K auf 3–5 reduzieren, wenn Token-成本 pro 请求 minimiert 将 sollen. | 阈值 e: RRF-Konstante K: Std 60. BM25-Gewicht: 0.0 = 仅 Vektor | 0.5 = Balance | 1.0 = 仅 Stichwort (Std: 0.50). Top-K: Std 5.

95. Lokaler Knowledge-Graph / Tripel-存储 (backend/knowledge_graph_store.py)

Problem: Flache 文本-Chunks erfassen 无 strukturierten Abhängigkeiten (wer verwaltet welches System 或 welche 组件 nutzt welchen 服务).

Mechanik: Extrahiert 与 speichert Subjekt-Prädikat-Objekt-Tripel 与 Konfidenzscore 在 SQLite 与 ermöglicht mehrstufige Graph-Traversierungen (Multi-Hop).

收益: 智能体 en erkennen System- 与 组织 szusammenhänge 关于 mehrere Ebenen hinweg.

Preis & Schattenseite (Trade-off): 自动 e Tripel-Extraktion kostet 在 存储 n ~500 Zusatz-Tokens. Ab > 2 Traversal-Hops explodiert Anzahl Nachbarknoten exponentiell.

Wann ausschalten? & 阈值 e: Auto-Extraktion ausschalten 在 reinem Kurzzeit-聊天 无 langfristigen 知识 saufbau; Hops auf maximal 2 begrenzen. | 阈值 e: Max Hops: 1 = direkte Nachbarn | 2 = optimal | >= 3 = 节点-Explosion! (Std: 2). Min. Konfidenz: Std 0.70.

96. 智能体 ic Corrective RAG / CRAG (backend/corrective_rag.py)

Problem: Klassisches RAG reicht irrelevante Suchtreffer ungeprüft an Sprachmodell weiter 与 provoziert dadurch Fehlantworten.

Mechanik: Bewertet jeden abgerufenen Dokument-Chunk gegen einen Relevanz-阈值 (0.45) 与 führt 在 schwachen Treffern 自动 ein Query-Rewrite 与 Synonym-Expansion durch.

收益: 筛选 t 知识 srauschen 在…之前 回答 generierung heraus 与 erhöht Trefferqualität 在 unklaren 问题 n.

Preis & Schattenseite (Trade-off): Jede Query-Umformulierung kostet 1 zusätzlichen LLM-Call (+1,5 bis +4 Sek. 延迟). Ist Relevanz-Schwelle zu hoch (> 0.70), dreht CRAG unnötige Extrarunden.

Wann ausschalten? & 阈值 e: Ausschalten, wenn 知识 sdatenbank klein 与 sauber gepflegt 是 或 schnelle 回答 zeiten 在…下 2 Sekunden gefordert sind. | 阈值 e: Relevanz-Schwelle: < 0.30 = lässt schwache 来源 n 通过 | > 0.70 = endlose Such-Schleifen (Std: 0.45). Max Rewrites: Std 2.

97. 2-Stufen Core-Memory 在…之后 MemGPT/Letta-Vorbild (backend/core_memory_manager.py)

Problem: Wichtige Nutzerpräferenzen 与 项目 ziele gehen verloren, sobald sie aus dem kurzlebigen 聊天-Fenster herausfallen.

Mechanik: Verankert editierbare Pinned-Working-Memory-Blöcke (<CORE_WORKING_MEMORY>) dauerhaft 在

System-Prompt, getrennt vom großen Archiv-存储.

收益: 担保 rt ein permanentes, vom 智能体 en selbst pflegbares 内核 gedächtnis 关于 beliebig viele 会话 en hinweg.

Preis & Schattenseite (Trade-off): Jeder angepinnte Block belegt 在 absolut jedem LLM-Aufruf dauerhaft Prompt-Tokens (10 Blöcke × 800 Zeichen ≈ 2.000 Tokens Grundlast pro 消息).

Wann ausschalten? & 阈值 e: Ausschalten 或 Block-Anzahl auf 3–5 reduzieren, wenn kleine 模型 e 与 engem Kontextfenster 或 knappe Token-预算 s genutzt werden. | 阈值 e: Max Blöcke: > 15 = hoher Token-Grundverbrauch (Std: 10). Max Zeichen/Block: < 300 = schneidet Regeln ab | > 1500 = verstopft Prompt (Std: 800).

98. 自动 es Citation- & 来源 n-Pinning (backend/citation_pinner.py)

Problem: Ohne direkte 来源 nverweise 是 针对 den Nutzer 不 überprüfbar, aus welchem Dokument eine Faktenaussage des 智能体 en stammt.

Mechanik: Ordnet Sätze 回答 per Token-Overlap den abgerufenen Quell-Dokumenten zu, fügt Inline-Belege ([来源: ID]) ein 与 prüft Mindest-Belegquote.

收益: Macht 每 Aussage revisionssicher nachvollziehbar 与 deckt unbelegte Behauptungen sofort auf.

Preis & Schattenseite (Trade-off): Erhöht den Output-Token-Verbrauch um ~10–15 %; eine zu hohe Mindest-Abdeckung (> 80 %) führt 在 kreativen 或 zusammenfassenden 问题 n zu Fehlwarnungen.

Wann ausschalten? & 阈值 e: Ausschalten 在 freiem Brainstorming, Code-Generierung 或 lockerem Alltags-聊天 无 Dokumentenbezug. | 阈值 e: Min. Abdeckung: < 30% = unbelegte Aussagen | > 85% = Fehlwarnungen 在 总结 en (Std: 60%).

15. 可靠执行与确定性 (持久执行)

99. Structured Outputs & JSON-Schema-担保 (backend/structured_output_enforcer.py)

Problem: Sprachmodelle liefern manchmal ungültiges JSON (Markdown-Rahmen, fehlende Klammern 或 falsche Datentypen), wodurch nachgelagerte Code-Pipelines abstürzen.

Mechanik: Repariert Syntaxfehler 在 Roh-String 自动 与 erzwingt 所有 Pflichtfelder 与 Datentypen des übergebenen 目标-Schemas.

收益: 担保 rt 100 % maschinenlesbare, typensichere JSON-Payloads 之间 智能体 en 与 Backend-接口 n.

Preis & Schattenseite (Trade-off): Jede Auto-Reparatur-Schleife kostet +1 bis +3 Sek. 延迟 与 zusätzliche Tokens. Strikte Schema-验证 lehnt 回答 en 也 wegen harmloser Zusatzfelder ab.

Wann ausschalten? & 阈值 e: Strikte 验证 ausschalten, wenn API-Endpunkte 或 Skripte 也 与 flexiblen 或 teilweisen JSON-Feldern umgehen können. | 阈值 e: Schema-检验: Strikt vs. Tolerant (Std: Strikt). Max Reparaturen: 1–2 = repariert 98 % aller Tippfehler | > 3 = verschwendet Tokens (Std: 2).

100. 持久执行 Checkpointing / Pause & Resume (backend/durable_checkpoint_manager.py)

Problem: Bricht ein 10-schrittiger Recherche- 或 Build-订单 在 步骤 8 通过 einen 新建 start ab, mussten bisher 所有 7 Vorarbeiten teuer wiederholt werden.

Mechanik: Sichert 在…之后 jedem abgeschlossenen 步骤 den kompletten Variablen- 与 Fortschrittszustand transaktional 在 SQLite.

收益: Unterbrochene Langläufer setzen exakt 在 letzten gültigen Checkpoint wieder auf, 无 Arbeit 或 Tokens

doppelt zu verbrauchen.

Preis & Schattenseite (Trade-off): Checkpointing 在 jedem einzelnen 步骤 (N = 1) erzeugt spürbare SQLite-Schreiblast. Auto-Resume 在 Booten 可 veraltete Tasks vom Vortag ungewollt wieder anfeuern.

Wann ausschalten? & 阈值 e: Auto-Resume ausschalten, wenn unterbrochene 任务 n 在…之后 einem 新建 start erst 手动 geprüft 与 freigegeben 将 sollen. | 阈值 e: Alle N 步骤 e: 1 = absolut absturzsicher, 更多 Disk-I/O | 3–5 = schont SSD (Std: 1). 归档: Std 14 Tage.

101. Idempotency-Keys & Tool-Call-Deduplizierung (backend/idempotency_guard.py)

Problem: Bei 网络-Retries 或 智能体 en-Schleifen besteht Gefahr, dass dieselbe 电子邮件, 发票 或 Webhook-Aktion mehrfach ausgelöst wird.

Mechanik: Vergibt 针对 jeden mutierenden Tool-Call einen SHA-256-Idempotency-Key 在 SQLite-Ledger 与 blockiert identische Folgeaufrufe 之内 TTL.

收益: Verhindert doppelte Außenwirkungen zuverlässig 在…之后 dem Vorbild moderner 付款 s-网关 s (Stripe-Prinzip).

Preis & Schattenseite (Trade-off): Wenn Nutzer exakt denselben Befehl 之内 Sperrfrist (默认 24 h) absichtlich ein zweites Mal ausführen will, blockiert Guard.

Wann ausschalten? & 阈值 e: TTL-Stunden auf 1 h senken 或 kurzzeitig ausschalten, wenn identische Aktionen 在 测试 betrieb mehrfach hintereinander ausgeführt 将 müssen. | 阈值 e: TTL (Stunden): < 1h = kurzer Duplikatschutz | > 48h = blockiert gewollte Wiederholungen (Std: 24h).

102. Dry-Run / Shadow-Mode 针对 智能体 en-Aufträge (backend/dry_run_simulator.py)

Problem: 新建 e 或 komplexe 智能体 en-Aufträge möchte man 在…之前 scharfen 执行 oft erst 无 风险 an echten 生产 sdaten testen.

Mechanik: Fängt 在 Shadow-Mode 所有 schreibenden 文件-, 数据库- 与 API-Befehle ab 与 protokolliert stattdessen einen detaillierten Mutations-Vorschauplan.

收益: Ermöglicht gefahrlose Trockenübungen 与 风险-审计 s 在…之前 endgültigen 批准.

Preis & Schattenseite (Trade-off): SOLANGE AKTIV, WERDEN KEINE ECHTEN DATEIEN GESCHRIEBEN UND KEINE ECHTEN APIS AUSGEFÜHRT! Vergisst man den Schalter, wirkt System 在…之后 außen hin 'tot'.

Wann ausschalten? & 阈值 e: Im normalen Produktivbetrieb zwingend AUSGESCHALTET lassen! Nur gezielt 针对 Trockenübungen (Dry-Runs) 与 安全 s-审计 s einschalten. | 阈值 e: Hauptschalter standardmäßig AUS (false). Nur 针对 isolierte 测试 läufe aktivieren.

103. Dead-Letter-Queue (DLQ) 与 Auto-Replay-Manager (backend/dead_letter_queue.py)

Problem: Fehlgeschlagene Webhooks 或 asynchrone Hintergrund-Jobs gehen 在 kurzzeitigen Drittanbieter-Ausfällen 无 Auffangbecken spurlos verloren.

Mechanik: 存储 t jeden gescheiterten Job samt Original-Payload 与 错误 grund 在 DLQ-表格 与 wiederholt ihn 与 exponentiellem Backoff.

收益: Kein 订单 geht verloren; fehlgeschlagene Tasks lassen sich jederzeit einzeln 或 gesammelt erneut abspielen.

Preis & Schattenseite (Trade-off): Bei dauerhaft fehlerhaften Aufträgen (z. B. ungültiger API-Key) verbrauchen Wiederholungsversuche unnötig 资源 n; zeitversetzte Replays 可 Aktionen verspätet auslösen.

Wann ausschalten? & 阈值 e: Auto-Replay ausschalten, wenn fehlgeschlagene Aufträge ausschließlich 手动 gesichtet 与 einzeln freigegeben 将 sollen. | 阈值 e: Max Retries: 1 = bricht 在 kurzem Ausfall ab | > 5 = staut tote

Jobs an (Std: 3). Backoff-Basis: Std 10s (10s -> 20s -> 40s).

16. 安全、数据保护与企业护栏

104. Bidirektionaler PII-Scrubber & Re-Hydration (backend/pii_scrubber.py)

Problem: Werden 电子邮件 s, Telefonnummern 或 IBANs 在 Klartext an externe Cloud-LLMs übertragen, drohen DSGVO-Verstöße 与 ungewollter Datenabfluss.

Mechanik: Ersetzt sensible Personendaten 在…之前 dem API-Aufruf lokal 通过 Platzhalter ([EMAIL_1], [IBAN_1]) 与 tauscht 本 在 eintreffenden 回答 wieder gegen echten 价值 e aus.

收益: Cloud-模型 e sehen niemals echte Personendaten, während Nutzer 在…之前 Ort dennoch vollständige Dokumente 与 回答 en erhält.

Preis & Schattenseite (Trade-off): Wenn LLM den Platzhalter 在 回答 text leicht abwandelt, schlägt Rückübersetzung (Rehydrierung) fehl. Zudem 可 Regex-Maskierung manchmal Code-Strings verfälschen.

Wann ausschalten? & 阈值 e: Ausschalten, wenn ausschließlich lokale 模型 e (Ollama) genutzt 将 或 LLM zwingend echten 电子邮件-Domains/Namen analysieren muss. | 阈值 e: Granulare Schalter 针对 电子邮件 s, Telefonnummern, IBANs 与 automatische Rehydrierung (alle standardmäßig aktiv).

105. Egress-防火墙 & Domain-Allowlisting 针对 智能体 en (backend/egress_firewall.py)

Problem: Über versteckte Prompt-Injections 在 Webseiten könnten 智能体 en angewiesen werden, interne Daten per HTTP-Request an fremde Angreifer-服务器 zu senden.

Mechanik: Prüft 每 ausgehende 目标-URL gegen konfigurierte Domain-Allowlist 与 blockiert unautorisierte Hosts sowie rohe IPv4-Adressen.

收益: Unterbindet Daten-Exfiltration auf 网络 ebene selbst 在 manipulierten 输入 n.

Preis & Schattenseite (Trade-off): Solange allowlist_only aktiv ist, 将 每 neue Webseite, API 或 文档, 不 explizit 在 Domain-Liste steht, 在 Web-Recherchen hart blockiert.

Wann ausschalten? & 阈值 e: Ausschalten (oder benötigte Domains ergänzen), wenn Research-智能体 frei 在 offenen Web recherchieren soll. | 阈值 e: Erlaubte Domains: Kommagetrennte Whitelist. Roh-IP-Blockade schützt 在…之前 direkten IP-Exfiltrationen.

106. Granulare 智能体 en-RBAC & 文件-Scope-Jails (backend/agent_rbac_jail.py)

Problem: Wenn 每 Sub-智能体 vollen Schreibzugriff auf gesamte 项目 verzeichnis besitzt, 可 ein Frontend-智能体 versehentlich Backend-内核 el-文件 en 或 .env-密钥 überschreiben.

Mechanik: Bindet 每 智能体 enrolle an eine feste 工具 liste 与 ein erlaubtes 目录-Jail (z. B. static/ 针对 Frontend-智能体 en).

收益: Setzt Prinzip minimaler 法律 e (Least Privilege) 针对 jeden einzelnen 流程 在 运行 ssystem durch.

Preis & Schattenseite (Trade-off): Verhindert, dass ein Administrator-智能体 legitim 配置 sdateien 之外 des 项目 ordners wartet 或 .env-模板 n direkt überprüft.

Wann ausschalten? & 阈值 e: Nur 针对 dedizierte Systemwartungs-任务 n kurzzeitig lockern; .env-保护 sollte 在 标准 albetrieb immer aktiv bleiben. | 阈值 e: Schalter 针对 Path-Jail, .env-Sperre 与 Systemordner-Sperre (/etc, /root, /proc).

107. Deterministischer Output-Linter & Policy-Checker (backend/output_policy_linter.py)

Problem: Sprachmodelle verfallen gelegentlich trotz System-Prompt 在 künstliche KI-Floskeln 或 generieren riskante Shell-Snippets.

Mechanik: Scannt den generierten 文本 在 Echtzeit mittels regulärer Ausdrücke, entfernt Sycophancy-Opener/AI-Slop 与 entschärft destruktive Befehle.

收益: Sichert sprachliche Präzision 与 technische 安全 无 zusätzlichen LLM-Prüfaufruf.

Preis & Schattenseite (Trade-off): Der Destruktiv-筛选 可 也 harmlose Aufräum-Skripte 或 gewollte 数据库-迁移 en blockieren, wenn darin Befehle 如 DROP 或 rm -rf vorkommen.

Wann ausschalten? & 阈值 e: Destruktiv-Blocker kurzzeitig ausschalten, wenn gezielt 服务器-Aufräumskripte 或 SQL-Reset-迁移 en erstellt werden. | 阈值 e: Schalter 针对 KI-Floskeln entfernen, Destruktive Befehle blockieren 与 Auto-Bereinigung.

108. Manipulationssichere Execution Receipts / SHA-256-Quittungen (backend/execution_receipt_ledger.py)

Problem: Für 合规-审计 s (EU AI Act / ISO 42001) reichen einfache 文本-Logs 不 aus, da sie nachträglich verändert 将 können.

Mechanik: Verkettet 每 Task-结算 (模型, Tokens, USD-成本, ausgeführte Tools) 与 dem SHA-256-Hash des Vorgängereintrags 在 einer Blockchain-ähnlichen Kette.

收益: Jede nachträgliche Manipulation an 成本- 或 决策 sprotokollen fällt 在 完整性 sprüfung sofort auf.

Preis & Schattenseite (Trade-off): Verursacht minimalen Rechen- 与 存储-Overhead (~2 ms 与 ~2 KB SQLite-存储 pro Task); Hash-Kette wächst 关于 Monate kontinuierlich an.

Wann ausschalten? & 阈值 e: Kann 在 Produktivbetrieb dauerhaft aktiviert bleiben; 仅 auf extrem speicherlimitierten Kleinstsystemen ausschalten. | 阈值 e: Hash-Verkettung (SHA-256) + detaillierte 成本-Aufschlüsselung pro Lauf.

17. 自优化、评估与群体智能

109. 自动 er Prompt-Optimizer 在…之后 DSPy/TextGrad-Prinzip (backend/prompt_optimizer.py)

Problem: Statische System-Prompts wiederholen dieselben 错误 immer wieder, solange kein Mensch 手动 nachbessert.

Mechanik: 价值 et reale 错误-Logs einer 角色 aus, erkennt systematische Schwächen 与 generiert gezielte Guardrail-Zusatzregeln 针对 den System-Prompt.

收益: 智能体 en lernen kontinuierlich aus gescheiterten Läufen 与 härten ihre Instruktionen selbstständig.

Preis & Schattenseite (Trade-off): Ist Auto-Apply aktiv, 可 eine fehlerhaft abgeleitete Regel Verhalten des 智能体 en dauerhaft verzerren (Prompt-Drift) 与 den System-Prompt aufblähen.

Wann ausschalten? & 阈值 e: Auto-Apply 在 Produktivbetrieb ausgeschaltet lassen, damit vorgeschlagene Prompt-补丁 es erst 在…之后 menschlicher 检验 übernommen werden. | 阈值 e: Ab 错误 n: 1 = überreagiert auf Einzelfehler | > 8 = lernt 很 träge (Std: 3). Max Regeln: > 10 = bläht System-Prompt auf (Std: 5).

110. Gold-默认 Regression-测试-Suite / Eval-Runner (backend/eval_regression_runner.py)

Problem: Eine vermeintlich kleine 变更 在 System-Prompt 或 模型 可 bestehende 安全 s- 或 Formatregeln unbemerkt verschlechtern.

Mechanik: Führt 在…之前 Übernahme neuer 设置 en automatisierte Gold-默认-测试 fälle aus 与 prüft Bestehensquote sowie verbotene Muster.

收益: Verhindert 质量 s-Regressionen 在 Produktivbetrieb 通过 ein objektives Quality-Gate.

Preis & Schattenseite (Trade-off): Jeder komplette Eval-Durchlauf feuert 所有 hinterlegten 测试-Prompts gegen LLM-API (kostet je 在…之后 测试 umfang ~5.000–25.000 Tokens 与 15–60 Sek. 期限).

Wann ausschalten? & 阈值 e: Deploy-Blockade ausschalten, wenn schnelle experimentelle Prompt-测试 s 无 Wartezeit durchgeführt werden. | 阈值 e: Min. Pass-Rate: < 75% = lässt Verschlechterungen 通过 | 100% = blockiert 已 在 kleinsten Abweichungen (Std: 90%).

111. Multi-Armed Bandit 模型-Router (backend/bandit_model_router.py)

Problem: Feste 模型-Zuweisungen reagieren 不 darauf, wenn ein Cloud-Provider plötzlich langsamer 将 或 erhöhte 错误 raten zeigt.

Mechanik: Bepunktet 每 模型 laufend 在…之后 realer Erfolgsquote (60 %) 与 Geschwindigkeit (40 %) 与 wählt dynamisch den besten Kandidaten.

收益: Maximiert 回答 qualität 与 Tempo 自动 anhand echter 期限-Telemetrie.

Preis & Schattenseite (Trade-off): Durch Explorationsrate (Epsilon, z. B. 0.15 = 15 %) testet Router absichtlich immer wieder alternative 模型 e aus, 在 Einzelfall langsamer sein können.

Wann ausschalten? & 阈值 e: Ausschalten (oder Epsilon auf 0.00 setzen), wenn 针对 jeden Lauf ein 100 % festes, vorhersagbares 模型 erzwungen 将 soll. | 阈值 e: Epsilon: 0.00 = nutzt stur beste 模型 | > 0.30 = probiert zu oft schwächere 模型 e (Std: 0.15). Erfolgs-/Latenz-Gewicht: Std 0.60 / 0.40.

112. Dynamisches Ephemeral-团队-Spawning (backend/ephemeral_team_spawner.py)

Problem: Ein einzelner Generalisten-智能体 是 在 großen Querschnittsaufgaben (架构 + UI + Security + QA) schnell überfordert.

Mechanik: Rekrutiert passend 至 Missions-Briefing eine temporäre Spezialisten-Task-Force 与 löst sie 在…之后 erledigter Mission wieder auf.

收益: Bündelt Spezialwissen bedarfsgerecht 无 dauerhaften 资源 n-Overhead.

Preis & Schattenseite (Trade-off): Ein 4-köpfiges Ad-hoc-Team vervielfacht den Token-Verbrauch (3x bis 5x) 与 benötigt zusätzlichen Koordinations-Overhead 之间 den 团队 mitgliedern.

Wann ausschalten? & 阈值 e: Ausschalten 在 einfachen Einzelaufgaben 或 knappem Token-预算; Max. 团队 größe 在 Alltag auf 3–4 begrenzen. | 阈值 e: Max 团队 größe: 2–4 = ideal | > 5 = hoher Kommunikations-Overhead & 5x Token-Verbrauch (Std: 4).

113. Live-资源 n-Protector & Host-Backpressure (backend/resource_backpressure.py)

Problem: Parallele 容器, Browser-审计 s 与 lokale 流程 e 可 在 Lastspitzen den Host-RAM 或 CPU überlasten (OOM-Kill).

Mechanik: Überwacht CPU-, RAM- 与 Disk-指标 en 在 Echtzeit 与 drosselt niedrig priorisierte Hintergrund-Tasks ab 85 % 利用率 automatisch.

收益: Schützt den 服务器 zuverlässig 在…之前 Abstürzen 通过 Überlastung 与 hält kritische 内核 prozesse jederzeit reaktionsfähig.

Preis & Schattenseite (Trade-off): Sind 阈值 e zu niedrig eingestellt (CPU < 70 % 或 RAM < 70 %), 将 Hintergrund-任务 n 已 在 normalen kurzen Lastspitzen ständig ausgebremst.

> **Wann ausschalten? & 阈值 e:** Nicht komplett ausschalten — stattdessen 在 starker Hardware CPU-/RAM-Schwellen auf 90 % anheben. | 阈值 e: Max CPU/RAM: < 70% = drosselt 在 normaler Arbeit | > 92% = OOM-Gefahr (Std: 85%). Min. freie Disk: Std 2.0 GB.

18. Säule 3: Wirtschaft & ERP (alpha) & 多租户套件

模块 / View	功能 sumfang & Technische Merkmale (v0.5.1)
Adressen & CRM + PocketBase-Sync	Zentraler Kontakt-, Debitoren- (10000+) 与 Kreditorenstamm (70000+), 360°-Kontakt-Drawer, Multi-Channel-Timeline, 日历/Wiedervorlagen sowie 链接 edIn- & Medien-Sync 关于 PocketBase (Ports 3005/8090/8092).
Artikel & 性能 en (ART)	Stammdatenkatalog ab ART-1001 针对 服务 leistungen, Stundensätze, Pauschalen, SaaS-Abonnements sowie physische Produkte 与 Lagerbestandsführung.
Vorgänge & Verträge	Durchgängige Belegkette 报价 (ANG-YYYY-NNNN) → 订单 (AB-YYYY-NNNN) inkl. gewichteter Deal-Pipeline, 营业额-预测 与 合同 s-期限 enradar (30/14/7-Tage-Alarme).
发票 swesen & DATEV	Ausgangs- 与 Eingangsbelege 在…之后 § 14 UStG, GiroCode-付款 s-QR, Finanz-Analytics, automatisches 催款 (Dunning Engine) 与 EXTF/CSV-Export 针对 DATEV.
Corporate & 租户 en-隔离	Mehrere eigene Unternehmungen (company_id) 与 physisch getrennten SQLite-数据库 en 在 WAL-Modus, eigenen Nummernkreisen, Briefbögen 与 模板 n-Editor.

19. 架构、安全与部署特性 (v0.5.0 – v0.5.1)

Feature / Subsystem	实现 & 运行 snutzen (v0.5.1)
模块 are Router-架构	Auslagerung 在 dedizierte FastAPI-Router 在…下 archon/routers/ (main.py 的 16.254 auf 1.875 Zeilen reduziert, -88 %) 与 zentrale 配置 sschicht archon/core/config.py.
Security Hardening (Archon Hardened)	Schließung 的 25 kritischen 与 68 hohen 审计-Befunden; Starlette Threadpool-Entlastung, gehärtete UI-DOM-路径 e 与 Klasse-A Docker-Admin-代理 针对 hermes_updater.
RBAC-用户 verwaltung (17.10)	Granulare 角色 n (admin, operator, viewer) 关于 backend/auth_service.py 与 100.000 Runden PBKDF2-HMAC-SHA256 密码-Hashing 与 Single-User/Multi-User-Modus.
2-Phasen Systemupdate (17.11)	立即 ige Anzeige lokaler 版本/HEAD via ?local_only=true (~0,25 s) plus asynchroner GitHub-Abgleich, Pre-更新 SQLite Online-Snapshots 与 1-Klick-Rollback.
AnythingLLM 代理 & Chroma Stack	Nahtlose Einbettung 的 AnythingLLM 在 中央图书馆 关于 /anythingllm-proxy inkl. /assets/* Fast-Path 与 lokalem ChromaDB Vektor-Index.
Portabler 1-Click-Installer	平台 übergreifende Erstinstallation 关于 setup.bat, setup.ps1 与 install.sh 与 docker-compose.yml, .env.example, INSTALL_AGENT.md 与 9-sprachiger GitHub-Doku.
4-Augen 质量 s-Gate & Code-沙箱	自动 e Zweitprüfung 在 Kanban 在…之前 Ticket-Abschluss sowie interaktive Code-沙箱 在 Showroom (4.4.1) 至 Live-Vorschau erzeugter Artefakte.

20. 按字母排序的术语表与索引

Alphabetisches Begriffsverzeichnis aller 内核 module, 架构 komponenten 与 Features (v0.5.1) 与 exakten 页nzahlen.

Feature / Begriff	Kurzbeschreibung	页(n)
3-Tier 成本-Routing	自动 e 模型 auswahl 在 LOW / MEDIUM / HIGH 在…之后 任务nkomplexität 与 预算.	—
4-Augen 质量 s-Gate	Unabhängige 检验 abgeschlossener Kanban-任务 n 通过 einen Revisor-智能体 en.	第 3, 16, 36
Anti-AI-Slop & STE100	Stil- 与 Wortfilter 至 Vermeidung unnötiger Floskeln 与 Einsparung 的 Output-Tokens.	—
AnythingLLM 代理	Lokale Einbettung 的 AnythingLLM + ChromaDB 关于 /anythingllm-proxy 在 中央图书馆.	第 3, 36
Approval Gate (HITL)	Menschlicher Prüfschritt 在…之前 externen 电子邮件 s, Buchungen 或 kritischen 文件 zugriffen.	第 3
Archon Hardened (v0.5.1)	安全 s- 与 Stabilitäts-Release 与 modularer Router-Struktur 与 geschlossenen 审计-Befunden.	第 3, 36
Bandit 模型-Router	Adaptiver Router (Multi-Armed Bandit) 至 dynamischen 模型 wahl 在… 之后 Trefferquote 与 成本.	第 35
Bot Screen (Web-Navigator)	Visueller 执行 smonitor 针对 Browser-Interaktionen via Playwright.	—
ChromaDB	Lokale Vektordatenbank 针对 semantisches Dokumenten-Retrieval 在 Säule 2.	第 36
Context-Compaction & LRU-K	存储 auslagerung 与 Komprimierung langer 聊天-Kontexte 在 Erreichen des RAM-Limits.	—
Corporate & 租户enfähigkeit	Physische 数据库-Trennung mehrerer Gesellschaften per company_id.	—
Corrective RAG (CRAG)	Selbstprüfende 知识 sabfrage 与 Relevanzbewertung 在…之前 回答generierung.	第 30
DAG-Scheduler	Zweistufige 执行 splanung abhängiger Teilaufgaben 与 parallelem Fan-Out / Fan-In.	第 15
DATEV-导出 & GiroCode	EXTF/CSV-Schnittstelle 针对 会计 sdaten 与 EPC-付款 s-QR auf 发票 en.	—
Dead-Letter-Queue (DLQ)	错误 warteschlange 与 automatischer Wiederholungssteuerung 针对 gescheiterte Jobs.	第 32
Decision Engine (55 框架)	战略 modul 针对 strukturierte Boardroom-Debatten 关于 9 Methodik-类别 n.	—
DSPy / 文本 Grad Optimizer	自动 e 改进 的 System-Prompts anhand realer Evaluations-指标 en.	—
Egress-防火墙	Ausgehende 网络- 与 Domain-Kontrolle 针对 智能体 en-Toolaufrufe.	第 33

Feature / Begriff	Kurzbeschreibung	页(n)
Ephemeral Code-沙箱	Flüchtige, isolierte 容器-期限 针对 sicheren Python- 与 Skriptcode.	第 3
ERP-Belegkette (ANG -> AB -> RE)	Verknüpfte 销售 s- 与 会计 sdokumente inkl. Pipeline-预测 与 催款.	—
Event-Mesh & Webhook-Trigger	事件 gesteuerte Verarbeitung eingehender Webhooks 与 文件夹 änderungen.	—
Evey / Hermes 插件 s	模块 are System-扩展 en 针对 存储, Delegation, Verifikation 与 接口 n.	—
Hermes Master Orchestrator	Zentrale 内核-Engine 针对 目标 zerlegung, Delegation 与 Synthese 在 守护进程 auf Port 8420.	第 13
Hybrid-Search (FTS5 + RRF)	Kombination aus BM25-Volltextsuche 与 Vektorsuche per Reciprocal Rank Fusion.	第 30
Idempotency-Keys & Dry-Run	保护 在…之前 doppelter Tool-执行 sowie Simulationsmodus 无 Schreibzugriff.	—
JEV Router (0-Token Intent)	Deterministischer Vorfilter 无 KI-成本 针对 eindeutige Kurzbefehle.	第 11
Knowledge-Graph	Lokaler Tripel-存储 针对 strukturierte Entitätsbeziehungen neben Vektordaten.	第 30
MCP (Model Context Protocol)	默认 isierte Treiberarchitektur 至 sicheren 工具 anbindung.	第 6, 16
OmniRoute AI 网关	Multi-Provider-网关 inkl. Free-Tier-Routing, Latency-Hedging 与 Eval-Arena.	—
PBKDF2 / Argon2id RBAC	角色 n- 与 密码-Absicherung 在 System Control Center (Tab 17.10).	—
PII-Scrubber & Re-Hydration	Bidirektionale Anonymisierung sensibler Personendaten 在…之前 externen LLM-Calls.	第 33
Pre-Flight 成本 schätzer	Vorabberechnung erwarteter Token-成本 inkl. Task-预算-Stopp 在 Überschreitung.	—
Pre-Mortem & 共识-Gate	Simulierter 错误-Stresstest 与 Mehrfach-智能体 en-表决 在…之前 kritischen 批准 n.	—
Research Lab	Mehrstufige Web- 与 来源 nrecherche 与 Evidenz-Triangulation.	第 3, 7, 24
Router-架构 (archon/routers)	模块 are Backend-Aufteilung 在 schlanke FastAPI-Router (-88 % 文件größe 在 main.py).	第 36
Semantic Caching	24h-Zwischenspeicherung inhaltlich ähnlicher Prompts 至 Token-Einsparung.	—
SLA-Verträge (Contract Enforcement)	Formale Übergabe- 与 质量 szusicherungen 之间 spezialisierten 智能体 en.	第 28
Sleep-Cycle Konsolidierung	Nächtliche Deduplizierung 与 Langzeit-Verdichtung des 智能体 en-记忆 ses.	—
SQLite WAL Connection-Pool	Schreibsicherer Multi-Thread-数据库 zugriff 关于 Write-Ahead Logging.	第 29

Feature / Begriff	Kurzbeschreibung	页(n)
System Control Center (11 Tabs)	Zentrale Administrationskonsole inkl. 用户/RBAC 与 2-Phasen Systemupdate.	第 9, 28
Token Guard & Loop-Killer	Echtzeit-保护 gegen Endlosschleifen (Jaccard-检验) 与 Token-Überschreitungen.	—
Watchdog & Deadlock-Supervisor	30-Sekunden-监控 hängender Tasks, 容器 与 automatischer Selbstheilung.	—
审计-Log (SHA-256 Hashkette)	Revisionssichere 日志 kette aller System- 与 智能体 enaktionen.	第 20
持久执行 & Time-Travel	Unterbrechbare Langläufe 与 SQLite-Checkpoints 与 schrittweisem Execution-Replay.	—